Telematics Platform User Guide

5G NAD Module Tier 1 Power Management for ConMod

Document Information	
Filename	TP_F101U-00039_User_Guide_5G_NAD_Tier1_Power_Management_ConMod
Status	Release
Revision Date	Apr 23, 2025
Version	03.00
Document ID	TP_F101U-00039

Standard Proprietary Terms

THE REPRODUCTION, DISTRIBUTION AND UTILIZATION OF THIS DOCUMENT AS WELL AS THE COMMUNICATION OF ITS CONTENTS TO OTHERS WITHOUT EXPRESS AUTHORIZATION IS PROHIBITED. OFFENDERS WILL BE HELD LIABLE FOR THE PAYMENT OF DAMAGES. ALL RIGHTS RESERVED IN THE EVENT OF THE GRANT OF A PATENT, UTILITY MODEL OR DESIGN.

© Continental Automotive GmbH 2019

Confidential and Proprietary

Table of contents

1 Introduction 6
1.1 Notational Conventions 6
2 Setup 7
2.1 Tools/Equipment Needed 8
2.2 Setup Instructions 8
2.3 Setup Troubleshooting 8
3 Features/Components 8
3.1 Module Power Up 8
3.1.1 Overview 8
3.1.1.1 Platform Content Overview 8
3.1.1.2 Product Expectations 9
3.1.1.3 Extending Platform Functionality 10
3.1.2 Setup 10
3.1.2.1 Tools/Equipment Needed 10
3.1.2.2 Setup Instructions 10
3.1.2.3 Setup Troubleshooting 11
3.1.3 Usage 11
3.1.3.1 Basic Instructions 11
3.1.3.2 Detailed Instructions 11
3.1.3.3 Testing/Error Reporting 12
3.2 Module Power Down 12
3.2.1 Overview 12
3.2.1.1 Platform Content Overview 12
3.2.1.2 Product Expectations 13
3.2.1.3 Extending Platform Functionality 13
3.2.2 Setup 13
3.2.2.1 Tools/Equipment Needed 13
3.2.2.2 Setup Instructions 13
3.2.2.3 Setup Troubleshooting 14
3.2.3 Usage 14
3.2.3.1 Basic Instructions 14
3.2.3.2 Detailed Instructions 14

© Continental Automotive GmbH 2019
Confidential and Proprietary

3.2.3.3 Testing/Error Reporting 18
3.3 Module Reset 18
3.3.1 Overview 18
3.3.1.1 Platform Content Overview 18
3.3.1.2 Product Expectations 18
3.3.1.3 Extending Platform Functionality 18
3.3.2 Setup 19
3.3.2.1 Tools/Equipment Needed 19
3.3.2.2 Setup Instructions 19
3.3.2.3 Setup Troubleshooting 19
3.3.3 Usage 19
3.3.3.1 Basic Instructions 19
3.3.3.2 Detailed Instructions 19
3.3.3.3 Testing/Error Reporting 21
3.4 Modem Power Interface 21
3.4.1 Overview 21
3.4.1.1 Platform Content Overview 21
3.4.1.2 Product Expectations 22
3.4.1.3 Extending Platform Functionality 22
3.4.2 Setup 22
3.4.2.1 Tools/Equipment Needed 22
3.4.2.2 Setup Instructions 22
3.4.2.3 Setup Troubleshooting 22
3.4.3 Usage 22
3.4.3.1 Basic Instructions 22
3.4.3.2 Detailed Instructions 22
3.4.3.3 Testing/Error Reporting 25
3.5 Low Power Mode – suspend/resume 25
3.5.1 Overview 25
3.5.1.1 Platform Content Overview 25
3.5.1.2 Product Expectations 25
3.5.1.3 Extending Platform Functionality 26
3.5.2 Setup 26
3.5.2.1 Tools/Equipment Needed 26
3.5.2.2 Setup Instructions 26

© Continental Automotive GmbH 2019

Confidential and Proprietary

3.5.2.3 Setup Troubleshooting 26
3.5.3 Usage 26
3.5.3.1 Basic Instructions 26
3.5.3.2 Detailed Instructions 27
3.5.3.3 Testing/Error Reporting 32
3.6 Packet Data during Low Power Mode 33
3.6.1 Overview 33
3.6.1.1 Platform Content Overview 33
3.6.1.2 Product Expectations 33
3.6.1.3 Extending Platform Functionality 33
3.6.2 Setup 33
3.6.2.1 Tools/Equipment Needed 33
3.6.2.2 Setup Instructions 34
3.6.2.3 Setup Troubleshooting 34
3.6.3 Usage 34
3.6.3.1 Basic Instructions 34
3.6.3.2 Detailed Instructions 34
3.6.3.3 Testing/Error Reporting 39
3.7 Power Related GPIO 39
3.7.1 Overview 39
3.7.1.1 Platform Content Overview 39
3.7.1.2 Product Expectations 39
3.7.1.3 Extending Platform Functionality 40
3.7.2 Setup 40
3.7.2.1 Tools/Equipment Needed 40
3.7.2.2 Setup Instructions 40
3.7.2.3 Setup Troubleshooting 40
3.7.3 Usage 40
3.7.3.1 Basic Instructions 40
3.7.3.2 Detailed Instructions 41
3.7.3.3 Testing/Error Reporting 49
4 Known Issues 49
5 FAQs 53
6 Troubleshooting/Tips and Tricks 53

© Continental Automotive GmbH 2019
Confidential and Proprietary

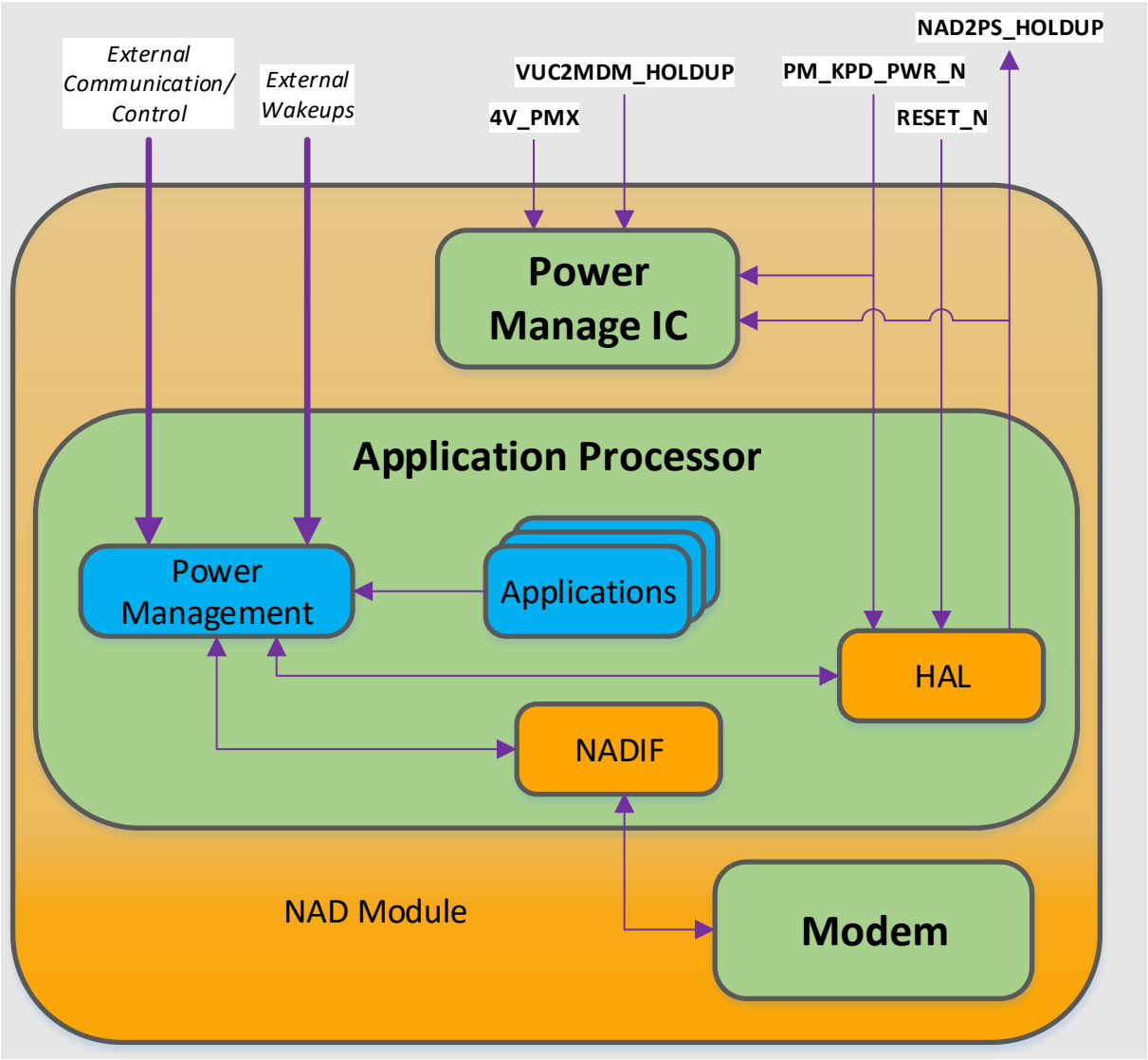
7 References 54
7.1 Abbreviations/Acronyms/Definitions 54
7.2 Platform Documentation 54
7.3 Website links 54
7.3.1 systemd 54
7.3.2 sysfs 54
8 Revision History 56

© Continental Automotive GmbH 2019
Confidential and Proprietary

1 Introduction

This document provides usage information for Tier1 supplier power management integration with the Continental 5G NAD Module. The Continental 5G NAD Module provides a 5G (5th generation) cellular modem with a Linux based application processor. This document provides detailed information relating to power management, including power management features/functionality expected from a Tier1 supplier using the 5G NAD Module.

Figure 11: Power Management System Context



1.1 Notational Conventions

Ctrl+x	*A sequence with a '+' indicates that you must press and hold one key or pointing device button while pressing another key or pointing device button*
[ ]	*Optional elements in a command*

© Continental Automotive GmbH 2019
Confidential and Proprietary

…..	*Indicates one of the following possibilities:* • *Additional optional arguments in a statement have been omitted* • *The preceding item or items can be repeated one or more times* • *Additional parameters, values, or other information can be entered*
-	*Line continuation: indicates that a command line or code line continues on the following line when at the end of a line*
Italic text	*Indicates important information, complete titles of manuals or variables*
`Mono-spaced`	*Example code or interactive screen display/output*
Numbers	*All numbers are assumed to be decimal unless otherwise noted. Non decimal radixes – binary, octal or hexadecimal – are explicitly indicated*
Underline blue text	*A link to an external reference (e.g., website, document)*

`Modules`
`|-> section`
`|-> item`
	A link to a section or item in the Telematics Platform html interface specification (Doxygen output from platform code headers)

Diagram Legend

2 Setup

HOWTO Be a Client of NADIF

See “Setup Instructions” and “Client Registration” in TP-F101U-00036.

HOWTO Access GPIO

1) Include HAL GPIO header file in client application source code.

```
#include “tphal_gpio_shared.h”
```

2) **Security**: Usage of HAL GPIO APIs require clients (users) to be a member of “gpio” group (see TP_F101U-00030).

API Description	Doxygen Path

Read (get) current value of an input GPIO	`Modules`
	`|-> Hardware Abstraction Layer (HAL)`
	`|-> HAL Library`
	`|-> General Purpose Input/Output (GPIO) Interface`

© Continental Automotive GmbH 2019
Confidential and Proprietary

	|-> **tpHAL_LowLevel_getGpio**
Register a callback to receive input GPIO status change notifications	Modules |-> Hardware Abstraction Layer (HAL) |-> HAL Library |-> General Purpose Input/Output (GPIO) Interface |-> **tpHAL_LowLevel_registerGpioChangeNotification**
Write (set) value of an output GPIO	Modules |-> Hardware Abstraction Layer (HAL) |-> HAL Library |-> General Purpose Input/Output (GPIO) Interface |-> **tpHAL_LowLevel_setGpio**

2.1 Tools/Equipment Needed

Not Applicable

2.2 Setup Instructions

Not Applicable

2.3 Setup Troubleshooting

Not Applicable

3 Features/Components

3.1 Module Power Up

3.1.1 Overview

3.1.1.1 Platform Content Overview

Module power up is initiated by the chipset automatically when `PM_KPD_PWR_N` GPIO input is asserted after power is applied to the module (see TP_F103U-00034). The bootloader will assert the `NAD2PS_HOLDUP` GPIO early in the module boot process to ensure that the module power supply is not turned off. Once the application processor boot up is complete, the Linux operating system is started and systemd is used for starting processes within the operating system. The NADIF service will power up the modem and establish a modem communication link.

systemd is an open source service manager that provides lazy initialization of services using D-Bus and/or sockets (start only when required) and aggressive parallelization (start as many services as possible). It also coordinates the start-up of services and boot modes supported by Telematics Platform.

All services/processes/applications are encapsulated in entities called “units”, which can be configured independently using configuration files with a `.service` extension. These configuration files also indicate unit

© Continental Automotive GmbH 2019
Confidential and Proprietary

order (i.e., start order – stop order is the reverse of start order) via dependencies. systemd automatically resolves unit dependencies to determine an appropriate unit start/stop order. The systemd units are typically installed in `/lib/systemd/system` path for Platform services, but any other location supported by systemd may be used as well.

Different units are grouped in entities called "targets", which are used as well-defined synchronization points during start-up. Targets are defined using a similar configuration file with a `.target` extension. Each target is considered started when all its units have been started successfully and considered stopped when all its units have been stopped successfully. Telematics Platform uses three different targets to represent the supported boot modes: `factory`, `recovery`, and `multi-user` (for normal mode). The `multi-user` target is then further divided into three target levels: `level1` (early), `level2` (mid), and `level3` (late). The `level1` target is started first, then `level2` target is started, and finally `level3` target is started. `level1` target should be reserved for services that are necessary for emergency calls or need to be started early due to timing constraints. In general, `level2` target should be used for services that have end user visibility. In general, `level3` target should be used for background operations, such as diagnostics and development tools.

Every target may have a directory containing symlinks to the units that need to be loaded by default as part of the target. These directories have the same base name as the target, with a `.wants` extension added. So, for `level1` target there is a `level1.target.wants` directory containing symlinks to the actual .service files for all the services that need to be started automatically as part of the "early" start up group. Similarly, there is a `level2.target.wants` directory containing symlinks to .service files for all of the services that need to be started automatically as part of the "mid" start up group and a `level3.target.wants` directory for all of the services that need to be started automatically as part of the "late" start up group.

For applications/services that are required on demand (i.e., not part of a start up group), systemd uses the bus activation feature to request start of the application when accessed the first time. The DBus logic is responsible for queueing the request and processing the request when the application establishes its service name.

Refer to https://www.freedesktop.org/wiki/Software/systemd/ for background and a more detailed description about systemd.

3.1.1.2 Product Expectations

1) Platform services are configured properly. With the term "properly", it is meant that the related service dependencies in terms of start up are taken into account. Note that the default platform service configuration that is provided should only be modified (e.g., moving a service from one start up group to another) with platform consultation.
2) It is expected from the product teams, i.e. SDK consumers, to handle boot up sequence based on product specific requirements. In other words, a product specific service is allowed to be started within platform services, however, related dependencies should be solved accordingly.
3) Additional services that need to be started during power up should be added to provided start up target groups (i.e., `level1.target`, `level2.target`, `level3.target`) as appropriate.

   Refer to https://www.freedesktop.org/software/systemd/man/systemd.service.html for more information about available options when creating a service file.

4) Additional services have proper symlinks added to the specific `target.wants` directory to be loaded/started appropriately.
5) Additional service files service dependencies list for start up (and shut down) avoiding circular dependencies. Note that no platform provided services should be dependent on any additional services.

Note: All services should not have an "after" dependency on any service that is included in a later start up group or a "before" dependency on any service that is included in an earlier start up group. This situation will cause the later group to be started early and may delay start up of critical services. A service may have a dependency on other services in the same group to ensure start

© Continental Automotive GmbH 2019

Confidential and Proprietary

up order within the group. A service should not need an "after" dependency for services in an earlier start up group, as those services should be started before the dependent service anyway. A service should not need a "before" dependency for services in a later start up group, as those services should only start after the dependent service anyway.

Example:

A new service "foo" is created. Service foo needs to start in the level2 start up group.

- A `foo.service` file is created for the "foo" service and placed with other service files.
- The "foo" service is added to the service list in `level2.target` file.
- A symlink to `foo.service` is added in the `level2.target.wants` directory.

Notes about "foo" service:

- The "foo" service will now be started automatically during power up at the same time as all other services in the `level2.target` file.
- The `foo.service` file should not have any "after" dependencies for services listed in `level3.target`. If "foo" service needs to be started after a `level3` service, "foo" needs to be moved to `level3` group.
- The `foo.service` file should not have any "before" dependencies for services listed in `level1.target`. If a `level1` service needs to be started after "foo" service, "foo" needs to be moved to `level1` group.
- The `foo.service` file may have dependencies on other services in `level2.target`. systemd will ensure that the start order for services within `level2` group meet the specified dependencies.
- Although `foo.service` could have "after" dependencies for services listed in `level1.target`, this is redundant and unnecessary. All services in `level1.target` will be started before any service in `level2.target` is started, including "foo" service.
- Although `foo.service` could have "before" dependencies for services listed in `level3.target`, this is redundant and unnecessary. All services in `level2.target`, including "foo" service, will be started before any service in `level3.target` is started.

6) Avoid adding services to the `level1` start up group as much as possible. The `level1` group should only contain services that are necessary for emergency calls or require early start up due to timing constraints (e.g., emergency call detection/initiation application, applications that handle emergency call data).

3.1.1.3 Extending Platform Functionality

Refer to https://www.freedesktop.org/software/systemd/man/systemd.service.html for information about creating service files and the different options available.

3.1.2 Setup

3.1.2.1 Tools/Equipment Needed

Not Applicable

3.1.2.2 Setup Instructions

In order to create an application that will start automatically and will be managed by systemd service manager, a service file needs to be installed in one of the locations supported by systemd and proper symlinks added to the appropriate `.wants` directory.

© Continental Automotive GmbH 2019
Confidential and Proprietary

Here is an example portion of a Makefile for installing application “foo” in `level2` start up group. A similar symlink may be created if needed for other targets (e.g., `factory`, `recovery`).

```
...
install:
    ...
    $(INSTALL) -m 644 foo.service $(DESTDIR)/lib/systemd/system
    Ln -s ../foo.service $(DESTDIR)/lib/systemd/system/level2.target.wants
```

3.1.2.3 Setup Troubleshooting

Not Applicable

3.1.3 Usage

3.1.3.1 Basic Instructions

Not Applicable

3.1.3.2 Detailed Instructions

Following are examples of different `.service` files to represent typical application/process scenarios.

<u>Example: Simple Service</u>

Following is an example of a simple service executing a process that will start in the `level2` group and keep running until an external termination or the process enters a failure state. There may be another service monitoring the health/status of this service and will attempt to recover the service when it enters a failure state. This service has no start up dependencies on other services.

```
[Unit]
Description=Simple

[Service]
ExecStart=/usr/sbin/simple-daemon
Restart=always

[Install]
WantedBy=level2.target
```

<u>Example: One shot service</u>

Following is an example of a service that just needs to run once and exit after execution. These types of services are most commonly used for initialization or clean up scripts or file system checkers. This service will start with other services in `level3` group and will end when it completes its operation.

```
[Unit]
Description=Init one shot

[Service]
Type=oneshot
ExecStart=/user/sbin/init-one-shot

[Install]
WantedBy=level3.target
```

<u>Example: Simple Service with dependencies</u>

© Continental Automotive GmbH 2019
Confidential and Proprietary

Dependencies are useful to delay the start of a specific service until after certain conditions are fulfilled. There are soft dependencies (e.g., Before, After), useful to adjust/control unit starting order, and hard dependencies (e.g., Requires), which require another unit to be running before starting.

If a unit listed under the required dependency fails to start, the dependent unit will also fail.

```
[Unit]
Description=Simple with dependencies
After=basic.target simple-depends-on.service
Requires=app.service

[Service]
ExecStart=/usr/sbin/simple-with-dependencies-daemon

[Install]
WantedBy=level2.target
```

3.1.3.3 Testing/Error Reporting

To get information about the status of an application, the following command can be used:

```
systemctl status <service name|pid>
```

Examples:

```
systemctl status foo
systemctl status 366
```

The output of this command shows the current status of the process, the last time it was started and, optionally, the reason for the failure and last lines of logs related to the application. This information is useful for application troubleshooting.

Example output for a running service

```
? foo.service - Foo
   Loaded: loaded (/lib/systemd/system/foo.service; enabled;vendor preset: enabled)
   Active: active (running) since Wed 2018-03-21 11:24:58 UTC; 4 days ago
 Main PID: 366 (foo)
   CGroup: /system.slice/foo.service
             366 /usr/sbin/foo-init
```

Example output for a failed service

```
? bar.service - Bar service
   Loaded: loaded (/lib/systemd/system/bar.service; disabled; vendor preset: enabled)
   Active: failed (Result: exit-code) since Wed 2018-03-21 11:24:51 UTC; 4 days ago
  Process: 153 ExecStart=/usr/bin/bar (code=exited, status=1/FAILURE)
 Main PID: 153 (code=exited, status=1/FAILURE)
```

3.2 Module Power Down

3.2.1 Overview

3.2.1.1 Platform Content Overview

The Telematics Platform provides services to support module power down operation, but does not support processing of power down triggers and sequencing. These are the power down related services provided by the platform:

© Continental Automotive GmbH 2019
Confidential and Proprietary

- Initiate modem power down via NADIF API with modem power down notification on completion.
- (internal) Initiate operating system power down via systemd-login final process power down sequence, with deassertion of `NAD2PS_HOLDUP` GPIO by Linux kernel when systemd final process power down sequence is complete.
- (external) systemd monitoring of `PM_KPD_PWR_N` GPIO to initiate final process power down sequence, with deassertion of `NAD2PS_HOLDUP` GPIO by Linux kernel when systemd final process power down sequence is complete.

Applications are stopped by systemd via `SIGTERM` signals and `SIGKILL` is used for specific cases (e.g., a configured timeout to stop the service via `SIGTERM` is reached). systemd also provides options to customize the stopping steps for each application. The order for stopping a service with dependencies on other services is the reverse of starting the service (i.e., all dependent services will be stopped first, then the requested service will be stopped).

Refer to https://www.freedesktop.org/wiki/Software/systemd/ for background and a more detailed description about systemd.

3.2.1.2 Product Expectations

1) Process module power down requests and determine when power down should be initiated.
2) Stop external services as part of power down processing (e.g., GNSS receiver, USB, WiFi, audio). Note that this could be built into the systemd `SIGTERM` process for each service, but it would be better to initiate external service power down prior to systemd service termination if the power down processing is complex (e.g., multiple steps) or is time consuming.
3) (recommended) Power down modem prior to initiating systemd process power down.
4) Ensure non-volatile storage is prepared for power down prior to power removal (e.g., flush pending writes, stop access).
5) Externally initiate a module power down via the `PM_KPD_PWR_N` GPIO when module does not respond to power down request in a timely manner.
6) Externally remove module power supply to power down the module only after module indicates it is safe to remove power (e.g., `NAD2PS_HOLDUP` GPIO deasserted) or when module does not respond to power down request via `PM_KPD_PWR_N` GPIO in a timely manner. Note that abrupt module power removal should be used as a “last resort” effort when all other attempts to power down the module have failed, as this could cause non-volatile memory corruption in the module

3.2.1.3 Extending Platform Functionality

Not Applicable

3.2.2 Setup

3.2.2.1 Tools/Equipment Needed

Not Applicable

3.2.2.2 Setup Instructions

NADIF APIs should be used for modem power control. A NADIF client (see TP_F101U-00036) is required for accessing NADIF APIs and receiving NADIF event information. The NADIF client controlling modem power should be aware of module power status for modem power down initiation.

© Continental Automotive GmbH 2019
Confidential and Proprietary

3.2.2.3 Setup Troubleshooting

Not Applicable

3.2.3 Usage

3.2.3.1 Basic Instructions

General module power down sequence:

1) Initiate modem power down via NADIF.
2) Wait for modem power down complete.
3) Stop external services to prepare for power down.
4) Prepare non-volatile memory for power down.
5) Initiate systemd power down via HAL.
6) (external) Wait for power down complete via `NAD2PS_HOLDUP` GPIO deassertion (see TP_F103U-00034)
7) (external) Remove module power supply.

3.2.3.2 Detailed Instructions

API/Type Description	Doxygen Path

API to change modem power state	`Modules`
	`|-> Modem`
	`|-> NADIF`
	`|-> Modem Status`
	`|-> NADIF_modem_set_operating_mode`

Modem set operating mode response notification	`Modules`
	`|-> Modem`
	`|-> NADIF`
	`|-> Modem Status`
	`|-> NADIF_SET_OPERATING_MODE_RESP`

API to request operating system power down	`Modules`
	`|-> Hardware Abstraction Layer (HAL)`
	`|-> HAL Library`
	`|-> System Interface`
	`|-> tpHAL_poweroffOperatingSystem`

Module power down can be requested/initiated internally (e.g., via internal applications) or externally (e.g., via an external power control module communicating with a module power application). Once power down is initiated, modem power down is performed first along with any application specific clean-up, such as preparing non-volatile memory for power removal and powering down peripherals. After modem power down and application clean-up is complete, the final operating system power down via systemd is initiated, either internally via a platform API or externally via `PM_KPD_PWR_N` GPIO. When the systemd power down process is complete, `NAD2PS_HOLDUP` GPIO is deasserted by the Linux kernel to indicate it is safe to remove power from the module. The power down sequence is illustrated in Figure 22.

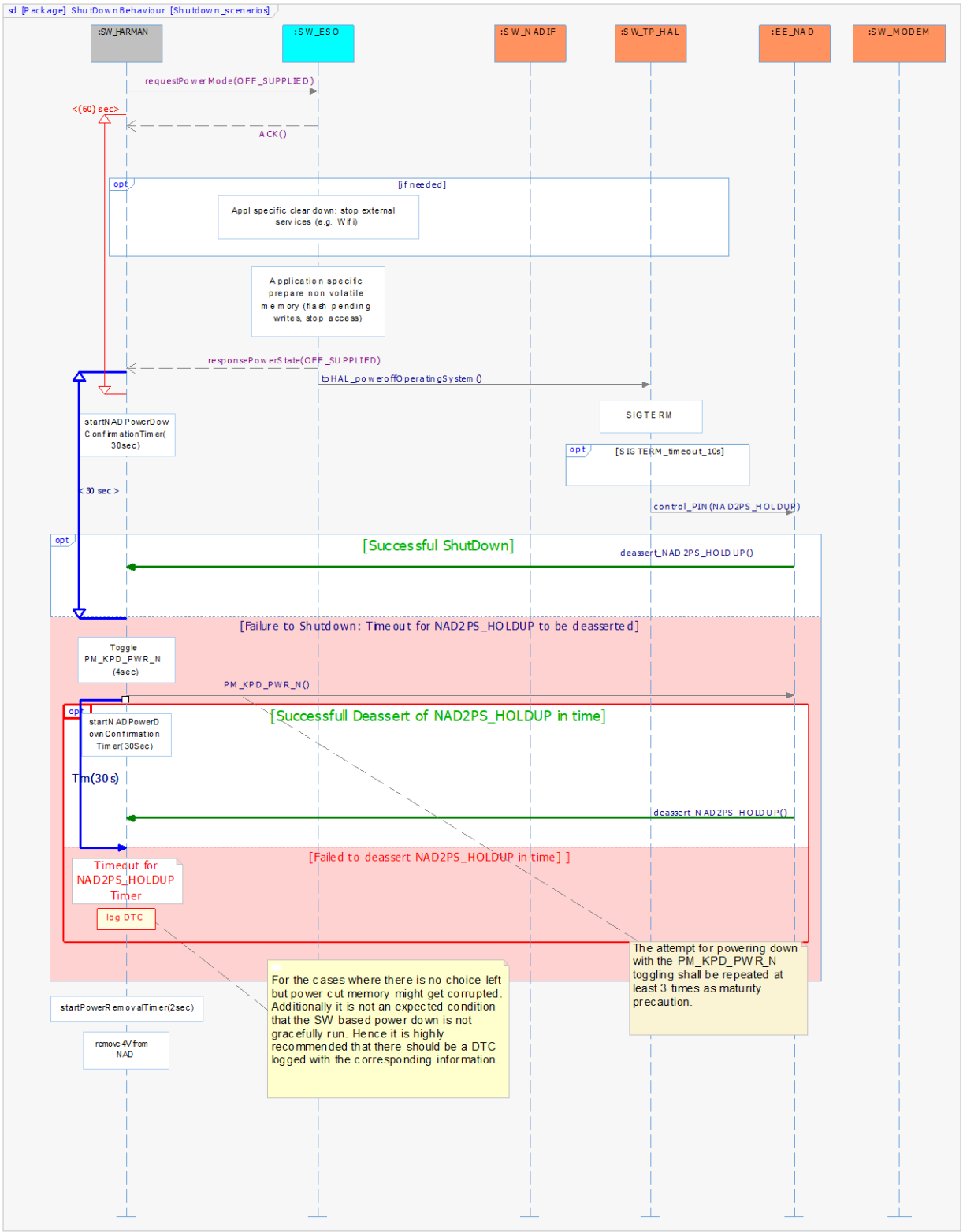
© Continental Automotive GmbH 2019
Confidential and Proprietary

Note: The application that invokes `tpHAL_poweroffOperatingSystem` API must be a member of "power" group.

© Continental Automotive GmbH 2019
Confidential and Proprietary

Figure 22: Module Power Down Sequence

© Continental Automotive GmbH 2019
Confidential and Proprietary



© Continental Automotive GmbH 2019
Confidential and Proprietary

3.2.3.3 Testing/Error Reporting

Module power down should be completed within a reasonable time period, otherwise unintended power consumption may occur. As such, it is recommended to time box critical steps in the power down sequence and force the next step when a timeout occurs. For example, if `NADIF_MODE_OFF` notification is not received during Modem Power Down sequence after some time (e.g., 15 seconds), prepare non-volatile memory for power off and continue power down via **`tpHAL_poweroffOperatingSystem`**.

Once Modem Power Down is initiated, the module must be power cycled to power the modem on (see "Module Reset"). If power down needs to be aborted after modem power down is complete, a module reset can be initiated instead of invoking **`tpHAL_poweroffOperatingSystem`**. Invocation of **`tpHAL_poweroffOperatingSystem`** is a point of no return. After **`tpHAL_poweroffOperatingSystem`** operation begins the module should be allowed to complete power down processing. Module power up can be performed without removing module power supply once power down processing is complete (e.g., `NAD2PS_HOLDUP` GPIO is deasserted).

3.3 Module Reset

3.3.1 Overview

3.3.1.1 Platform Content Overview

The Telematics Platform provides services to support graceful module reset (i.e., module power down followed by module power up), but does not support processing of reset triggers and sequencing. These are the module reset related services provided by the platform:

- Initiate modem power down via NADIF API with modem power down notification on completion.
- Initiate operating system reset via systemd process control.
- systemd monitoring of `RESET_N` GPIO for uncontrolled module reset (potential modem ungraceful shutdown).

As with Module Power Down, applications are stopped by systemd in reverse order of starting to prepare for the reset. Once applications have been stopped, an application processor reset is triggered instead of a power down stop. The processor reset will perform Module Power Up as would be performed with `PM_KPD_PWR_N` GPIO input assertion.

3.3.1.2 Product Expectations

1) Process module reset requests and determine when module reset should be initiated.
2) Perform periodic module resets for modem and Linux AP service stability (see 3.3.3.2 for recommended timing)
3) Prepare external services for module reset (e.g., GNSS receiver, USB, WiFi, audio).
4) (recommended) Power down modem prior to initiating systemd reset.
5) Externally initiate a module reset via the `RESET_N` GPIO when module does not respond to controlled reset request in a timely manner. Note that the modem may not be gracefully shut down, so this method of module reset should be used sparingly.
6) Externally remove module power supply to power down the module when module does not respond to reset request via RESET_N GPIO in a timely manner. Note that abrupt module power removal should be used as a "last resort" effort when all other attempts to reset the module have failed, as this could cause non-volatile memory corruption in the module.

3.3.1.3 Extending Platform Functionality

Not Applicable

© Continental Automotive GmbH 2019

Confidential and Proprietary

3.3.2 Setup

3.3.2.1 Tools/Equipment Needed

Not Applicable

3.3.2.2 Setup Instructions

NADIF APIs should be used for modem power control. A NADIF client (see TP_F101U-00036) is required for accessing NADIF APIs and receiving NADIF event information. The NADIF client controlling modem power should be aware of module power status for modem power down initiation.

3.3.2.3 Setup Troubleshooting

Not Applicable

3.3.3 Usage

3.3.3.1 Basic Instructions

General module reset sequence:

1) Initiate modem power down via NADIF.
2) Wait for modem power down complete,
3) Prepare external services for module reset (e.g., notification that reset will occur so module will be unavailable for a period of time, power down peripheral).
4) Initiate systemd module reset via HAL.
5) (external) Wait for module power down then power up to restore module access (e.g., communication link).

3.3.3.2 Detailed Instructions

API/Type Description	Doxygen Path

API to change modem power state	`Modules`
	`|-> Modem`
	`|-> NADIF`
	`|-> Modem Status`
	`|-> NADIF_modem_set_operating_mode`

Modem set operating mode response notification	`Modules`
	`|-> Modem`
	`|-> NADIF`
	`|-> Modem Status`
	`|-> NADIF_SET_OPERATING_MODE_RESP`

API to request operating system reset	`Modules`
	`|-> Hardware Abstraction Layer (HAL)`
	`|-> HAL Library`
	`|-> System Interface`

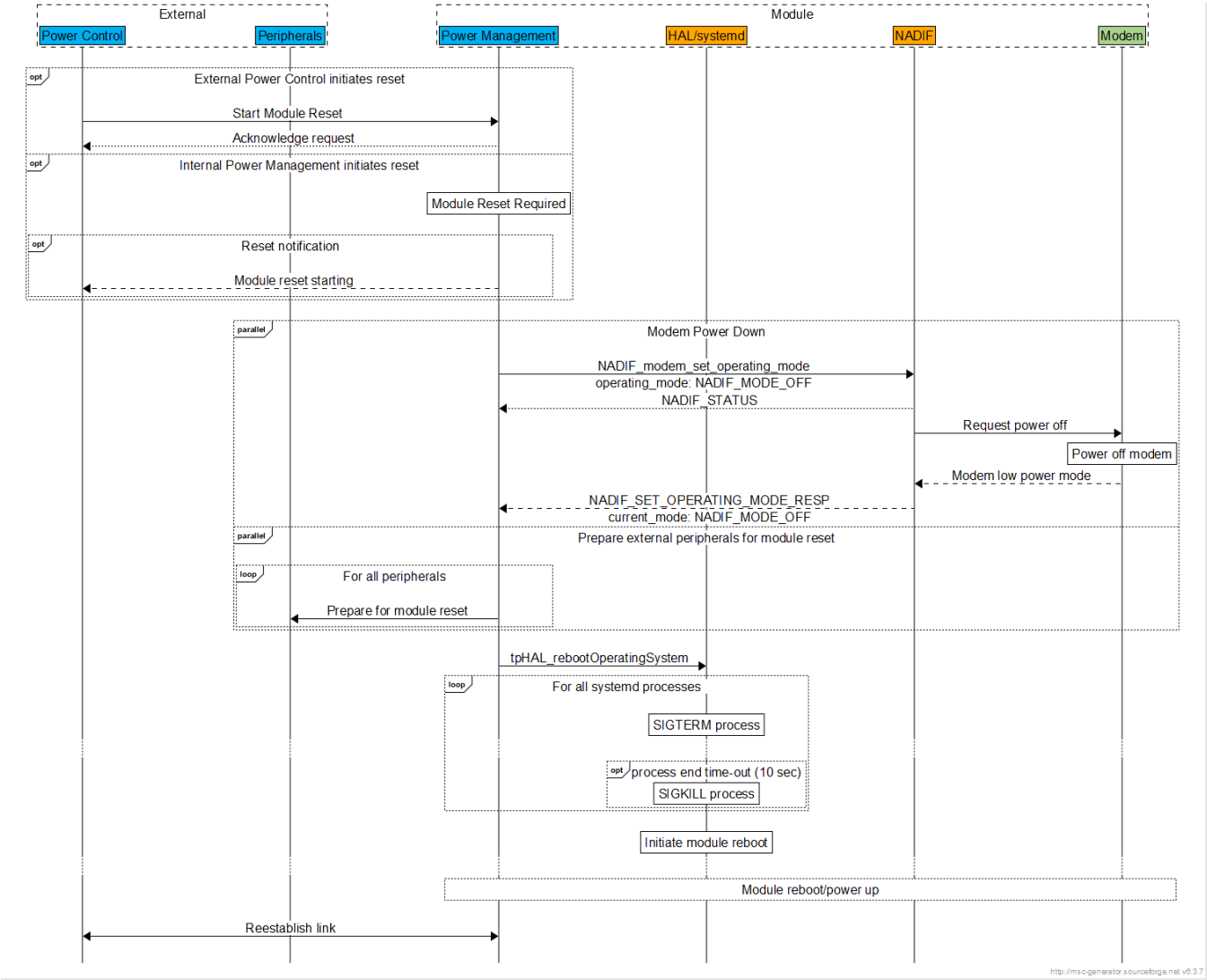
© Continental Automotive GmbH 2019
Confidential and Proprietary

	|-> `tpHAL_rebootOperatingSystem`

Module reset can be requested/initiated internally (e.g., via internal applications) or externally (e.g., via an external power control module communicating with a module power application). Once reset is initiated, modem power down is performed first along with any application specific clean-up, such as preparing external peripherals for the module reset. After modem power down and application clean-up is complete, the operating system reboot (reset) is initiated, preferably internally via a platform API (can be initiated externally via `RESET_N` GPIO). When the systemd process power down is complete, the application processor reboot is initiated to restart the module. The regular module reset sequence is illustrated in Figure 33.

Note: The application that invokes `tpHAL_rebootOperatingSystem` API must be a member of "power" group.

Figure 33: Module Reset Sequence



Periodic Module Reset

© Continental Automotive GmbH 2019
Confidential and Proprietary

Some cellular service providers require that the cellular modem is reset periodically (e.g., once every 24 hours) for embedded modules. Additionally, a periodic reset of the NAD module helps ensure proper module operation, as there are many vehicle usage scenarios where the module would not be reset for extended periods of time. As such, an algorithm should be implemented in the Tier 1 power management application to trigger a module reset regularly. The module reset triggered in this manner would be considered as internally initiated reset in Figure 33.

A periodic module reset algorithm can be fairly simple. A very simple algorithm (which is not recommended) would be to start a timer when the module powers up and reset the module when the timer expires. A simplistic algorithm, though, has some serious limitations, including resetting when the module is busy with other activities or resetting when the vehicle ignition is on (module should be available for user interaction). So, development of a more complex algorithm is strongly recommended.

Some things to consider when developing a periodic/regular module reset algorithm:

- Module reset period should be between 12 and 24 hours, inclusive. This ensures the module is reset on a regular basis that will meet cellular service provider requirements, but also prevent frequent module resets. A 24 hour reset period is recommended.
- Avoid resetting the module when ignition is on. It is highly unlikely that the ignition will be on in a vehicle for 12 hours or longer, so there should be opportunities for module reset when ignition is off.
- Include a delay after ignition is turned off before triggering a module reset. This will eliminate resets that are triggered when the vehicle occupant is out of the vehicle for a short period of time.
- Always perform a module reset shortly after transitioning from ignition on to ignition off. This will force the modem to perform a cellular system scan/selection, so the modem will be acquire the best service available during Low Power Mode.
- Reset the periodic reset timer whenever a module power cycle occurs. Examples: module is power cycled by an external power control module, module reset performed shortly after transitioning to ignition off, other internal triggers initiating a module reset.
- Ensure that the module is reset periodically during extended Low Power Mode periods. In other words, the periodic module reset timer should be running when in Low Power Mode and a module reset should be performed when the timer expires. For example, if the periodic module reset timer is 24 hours, the module should be reset once every 24 hours when ignition is off for longer than 24 hours (e.g., vehicle is used as a work commute vehicle, so is typically turned off over the weekend – reset the module once every 24 hours during the weekend).
- If possible, try to perform module resets at times when the vehicle is normally not in use, such as in the middle of the night for most vehicles (or middle of the day for a vehicle that is used at night).

3.3.3.3 Testing/Error Reporting

Module reset should be completed within a reasonable time period, otherwise an unrecoverable module error may have occurred. As such, it is recommended to time box the module reset and link reestablishment. If the module reset process does not complete then force a module reset or power cycle (i.e., remove module power, reapply power and initiate power up). Note that the time to complete the reset must account for module power down time and module power up time.

3.4 Modem Power Interface

3.4.1 Overview

3.4.1.1 Platform Content Overview

The NADIF module within the Telematics Platform provides APIs that can be used for modem power state and modem wakeup event access. One of the modem power state APIs, `NADIF_modem_set_operating_mode`, has

© Continental Automotive GmbH 2019
Confidential and Proprietary

already been mentioned in Module Power Down and Module Reset. These are the modem power interface related services provided by the platform:

- Request/set modem power state.
- Unsolicited modem power state change notification.
- Query current modem power state.
- Set modem events that will trigger a wakeup event notification.
- Unsolicited wakeup event notification.
- Query modem event(s) that caused a wakeup event.

3.4.1.2 Product Expectations

NADIF APIs for modem power interface can be used for modem power control and wakeup event control/processing. A product application must be registered as a NADIF client (see TP_F101U-00036) in order to access NADIF APIs and receive NADIF notifications for command responses and modem events.

3.4.1.3 Extending Platform Functionality

Not Applicable

3.4.2 Setup

3.4.2.1 Tools/Equipment Needed

Not Applicable

3.4.2.2 Setup Instructions

A NADIF client (see TP_F101U-00036) is required for accessing NADIF APIs and receiving NADIF event information.

3.4.2.3 Setup Troubleshooting

Not Applicable

3.4.3 Usage

3.4.3.1 Basic Instructions

Not Applicable

3.4.3.2 Detailed Instructions

The APIs and types detailed in this section of the document can be found in Doxygen with the following format:

Main Doxygen Path

```
|-> Doxygen subpath
    |-> Name of Data Type or API
```

Where *Main Doxygen Path* is

```
Modules
|-> Modem
    |-> NADIF
```


© Continental Automotive GmbH 2019
Confidential and Proprietary

API/Type Description	Doxygen Path
API to change modem power state	*Main Doxygen Path* |-> Modem Status |-> **NADIF_modem_set_operating_mode**
Modem set operating mode response notification	*Main Doxygen Path* |-> Modem Status |-> **NADIF_SET_OPERATING_MODE_RESP**
Unsolicited modem power state change notification	*Main Doxygen Path* |-> Modem Status/Notifications |-> **NADIF_FEAT_STATUS_OPER_MODE**
API to query current modem power state	*Main Doxygen Path* |-> Modem Status |-> **NADIF_modem_get_operating_mode**
Modem get operating mode response notification	*Main Doxygen Path* |-> Modem Status |-> **NADIF_GET_OPERATING_MODE_RESP**
API to set modem events that will trigger a wakeup event notification	*Main Doxygen Path* |-> Modem Status |-> **NADIF_modem_set_wakeup_reason**
Modem set wakeup events response notification	*Main Doxygen Path* |-> NADIF Common Types |-> **NADIF_RESULT**
Unsolicited modem wakeup event notification	*Main Doxygen Path* |-> Modem Status/Notifications |-> **NADIF_FEAT_STATUS_WAKEUP**
API to query modem event(s) that caused a wakeup event	*Main Doxygen Path* |-> Modem Status |-> **NADIF_get_wakeup_reason**
Modem get wakeup reason response notification	*Main Doxygen Path* |-> Modem Status |-> **NADIF_GET_WAKEUP_REASON_RESP**

© Continental Automotive GmbH 2019
Confidential and Proprietary

The **NADIF_modem_set_operating_mode** API allows NADIF clients to change the modem power mode. NADIF will provide status of the request via **NADIF_SET_OPERATING_MODE_RESP** notification. Operating modes that can be requested with this API:

- NADIF_MODE_ON: Used to return the modem to normal operating mode (e.g., RF active, searching/acquiring/acquired cellular service) from NADIF_MODE_LOW mode. Modem operating mode after power cycle will be NADIF_MODE_ON.
- NADIF_MODE_OFF: Used to initiate a modem power down to prepare for Module Power Down or Module Reset. Modem operating mode after power cycle will be NADIF_MODE_ON when the current mode is NADIF_MODE_ON or NADIF_MODE_LOW or will be NADIF_MODE_LOW when the current mode is NADIF_MODE_OFFLINE.
- NADIF_MODE_LOW: Used to request airplane mode (i.e., RF disabled). Modem operating mode after power cycle will be NADIF_MODE_ON.
- NADIF_MODE_OFFLINE: Used to request RF disabled until modem power cycled (power cycle required to exit mode). Modem operating mode after power cycle will be NADIF_MODE_LOW (i.e., RF disabled). This mode is typically used to request airplane mode to be maintained after a modem power cycle.
- NADIF_MODE_NO_SERVICE_RECOVER: Used to request modem rescan for service after the modem has been in a no service condition for an extended time period. Can only be requested when modem operating mode is NADIF_MODE_ON and modem operating mode remains NADIF_MODE_ON.

When the modem operating mode changes state, registered NADIF clients will receive an unsolicited **NADIF_FEAT_STATUS_OPER_MODE** notification providing the new modem operating mode. The current modem operating mode can also be queried via **NADIF_modem_get_operating_mode** API. NADIF will provide the modem operating mode via **NADIF_GET_OPERATING_MODE_RESP** notification. Modem operating modes that can be provided in either notification:

- NADIF_MODE_ON: Modem normal operating mode. RF is enabled and the modem is either searching for cellular service, acquiring cellular service, has acquired cellular service, or has determined no cellular service available. This is the typical modem operating mode.
- NADIF_MODE_OFF: Modem power off processing complete, modem ready for power removal.
- NADIF_MODE_LOW: Modem currently in airplane mode (i.e., RF disabled).
- NADIF_MODE_OFFLINE: Modem currently in airplane mode (i.e., RF disabled) and exiting airplane mode cannot be requested. This mode will remain until the modem is power cycled, after which the mode will be NADIF_MODE_LOW.

HOWTO enter airplane mode (e.g., cellular RF off)

Invoke **NADIF_modem_set_operating_mode** API requesting to set modem operating mode to NADIF_MODE_LOW when modem operating mode is NADIF_MODE_ON.

HOWTO exit airplane mode (e.g., cellular RF on)

Invoke **NADIF_modem_set_operating_mode** API requesting to set modem operating mode to NADIF_MODE_ON when modem operating mode is NADIF_MODE_LOW.

HOWTO force airplane mode on next modem power cycle

Invoke **NADIF_modem_set_operating_mode** API requesting to set modem operating mode to NADIF_MODE_OFFLINE when modem operating mode is NADIF_MODE_ON. Note that the mode will be NADIF_MODE_OFFLINE until the modem is powered down. Mode will be NADIF_MODE_LOW on next modem power cycle.

HOWTO power down modem

© Continental Automotive GmbH 2019
Confidential and Proprietary

Invoke **NADIF_modem_set_operating_mode** API requesting to set modem operating mode to NADIF_MODE_OFF when modem operating mode is NADIF_MODE_ON, NADIF_MODE_LOW, or NADIF_MODE_OFFLINE.

HOWTO query current modem power mode

Invoke **NADIF_modem_get_operating_mode** API. The current modem power mode will be provided in a **NADIF_GET_OPERATING_MODE_RESP** notification.

The modem wakeup event related APIs and notifications (**NADIF_modem_set_wakeup_reason** & **NADIF_RESULT**, **NADIF_modem_get_wakeup_reason** & **NADIF_GET_WAKEUP_REASON_RESP**, and **NADIF_FEAT_STATUS_WAKEUP**) are typically used for module Low Power Mode. The **NADIF_modem_set_wakeup_reason** API is used to request modem events that will trigger a modem wakeup event notification prior to suspending the application processor. When one of the enabled modem wakeup events occurs, the modem sends a wakeup notification message to NADIF, causing application processor operation to resume. NADIF then provides a **NADIF_FEAT_STATUS_WAKEUP** notification to registered NADIF clients to indicate the modem event(s) that caused the wakeup. A NADIF client then uses **NADIF_modem_get_wakeup_reason** API to query the modem wakeup events and clear the pending wakeup notification. Once the wakeup event is processed, the application processor can be suspended again until the next enabled modem wakeup event occurs. See Low Power Mode for additional information.

3.4.3.3 Testing/Error Reporting

Not Applicable

3.5 Low Power Mode – suspend/resume

3.5.1 Overview

3.5.1.1 Platform Content Overview

The Telematics Platform provides services to support module low power mode operation, but leaves the processing of the low power mode triggers and in-TCU-wakeup event generation to the product teams or customer. Module low power mode is a mode where the application processor in the module is suspended to reduce power consumption, but the modem remains powered on to monitor for cellular events to wake up the application processor (e.g., SMS message received). These are the low power mode related services provided by the platform:

- Enable modem events that will wakeup application processor while suspended via NADIF API.
- Query modem event(s) that triggered an application processor wakeup via NADIF API and clear wakeup event(s).
- Unsolicited NADIF notification of modem event(s) that triggered an application processor wakeup.
- Initiate application processor (operating system) suspend via HAL API.

3.5.1.2 Product Expectations

1) Process module low power mode requests and determine when application processor suspend should be initiated.
2) Enable modem events that should trigger application processor wakeup prior to application processor suspend.
3) Prepare external services for application processor suspend/low power mode (e.g., disable/turn off GNSS receiver by de-registering all client applications from location_hal_daemon service via TelSDK API, disable/turn off USB, turn off WiFi, disable audio) prior to application processor suspend.

© Continental Automotive GmbH 2019
Confidential and Proprietary

4) Unload any drivers that do not function correctly through or after a suspend/resume cycle prior to application processor suspend.
5) Enable external wakeup sources as appropriate (e.g., enable GPIO interrupts, serial bus interrupt/wake-up) prior to application processor suspend.
6) Disable unsolicited modem events reporting before suspend.
7) Initiate application processor suspend after suspend preparations complete.
8) Process application processor wakeup events after operation resumes (e.g., when exiting suspend).
9) Restart/reenable external services after resume as appropriate.
10) Reload any unloaded drivers after resume as appropriate.
11) Provide low power mode timing (e.g., when to transition from Low Power Mode to “Off” Mode), either via an external timer (e.g., when external timer expires it triggers an external wakeup and informs Power Management to power down the module) or an internal system timer (e.g., Linux system timer).
12) Disable unnecessary modem events that trigger application processor wakeup when application processor not suspended and not preparing for suspend.

3.5.1.3 Extending Platform Functionality

Not Applicable

3.5.2 Setup

3.5.2.1 Tools/Equipment Needed

Not Applicable

3.5.2.2 Setup Instructions

NADIF APIs should be used for modem wakeup event access. A NADIF client (see TP_F101U-00036) is required for accessing NADIF APIs and receiving NADIF event information. Only one NADIF client should control and process modem wakeup events. This single client should disseminate wakeup information to other application components as necessary.

3.5.2.3 Setup Troubleshooting

Not Applicable

3.5.3 Usage

3.5.3.1 Basic Instructions

General module low power entry/exit (suspend/resume) sequence:

1) Enable modem events that will wake up application processor while suspended via NADIF.
2) Prepare external services for application processor suspend/low power mode (e.g., disable/turn off peripherals as appropriate).
3) Enable external application processor wake up sources (e.g., enable interrupts).
4) Disable unsolicited modem events reporting before suspend.
5) Request application processor/operating system suspend.
6) Upon resume, determine wake up source/reason.
7) If wake up source is modem, query modem wake up event(s) via NADIF. This will also clear the wake up event trigger in the modem.
8) After processing wake up source/reason, determine whether to stay awake or go back to suspend.
9) Disable unnecessary modem events that will wake up application processor if staying awake.
10) Reenable external services as appropriate if staying awake.

© Continental Automotive GmbH 2019
Confidential and Proprietary

3.5.3.2 Detailed Instructions

API/Type Description	Doxygen Path
API to set modem events that will trigger a wakeup event notification	Modules |-> Modem |-> NADIF |-> Modem Status |-> **NADIF_modem_set_wakeup_reason**
Modem set wakeup events response notification	Modules |-> Modem |-> NADIF |-> NADIF Common Types |-> **NADIF_RESULT**
Unsolicited modem wakeup event notification	Modules |-> Modem |-> NADIF |-> Modem Status/Notifications |-> **NADIF_FEAT_STATUS_WAKEUP**
API to query modem event(s) that caused a processor wake up	Modules |-> Modem |-> NADIF |-> Modem Status |-> **NADIF_get_wakeup_reason**
Modem get wakeup reason response notification	Modules |-> Modem |-> NADIF |-> Modem Status |-> **NADIF_GET_WAKEUP_REASON_RESP**
API to enable/disable event report on Modem	Modules |-> Modem |-> NADIF |-> Control |->**NADIF_set_event_report**
API to get the last known reason(s) for resuming from suspended state	Modules |-> Hardware Abstraction Layer (HAL)

© Continental Automotive GmbH 2019
Confidential and Proprietary

	`|-> HAL Library`
	`|-> System Interface`
	`|-> tpHAL_getLastResumeReason`

API to request operating system suspend	`Modules`
	`|-> Hardware Abstraction Layer (HAL)`
	`|-> HAL Library`
	`|-> System Interface`
	`|-> tpHAL_suspendOperatingSystem`

Full list of supported resume reasons for HAL API `tpHAL_getLastResumeReason`:

Enum	Value	Description
E_RESUME_CAUSE_UNKNOWN	0	Unknown “spurios” reason for resume
E_RESUME_CAUSE_SUSPENDED_ABORTED	1	Suspend was aborted
E_RESUME_CAUSE_MODEM_CORE	2	Modem core caused resume
E_RESUME_CAUSE_GPIO	3	GPIO caused resume
E_RESUME_CAUSE_PM_CORE	4	Power Management core caused resume
E_RESUME_CAUSE_POWER_KEY	5	Power On key caused resume
E_RESUME_CAUSE_RESET_KEY	6	Reset key caused resume
E_RESUME_CAUSE_USB_VBUS	7	USB VBUS caused resume
E_RESUME_CAUSE_THERMAL	8	Thermal caused resume
E_RESUME_CAUSE_TIMER	9	Timer expiration caused resume
E_RESUME_CAUSE_DATA	10	Packet data wakeup triggered
E_RESUME_CAUSE_MODEM_CRASH	11	Modem crash event caused resume
E_RESUME_CAUSE_MODEM_CORRUPT	12	Modem identified as corrupt caused resume

Module low power mode (application processor suspend) can be requested/initiated internally (e.g., via internal applications) or externally (e.g., via an external power control module communicating with a module power application). Once low power mode is initiated, the following preparations may be taken as appropriate prior to requesting operating system suspend:

- Enable modem events that will trigger an application processor wake up (resume) from suspend. The NADIF API `NADIF_modem_set_wakeup_reason` is used for this purpose. The `wakeup_reasons` input parameter is a bitmask of wakeup events that can be enabled. If a wakeup event should be enabled, the corresponding bit in the request should be set to 1, otherwise the bit should be 0.

> **Example**: Enable processor wake up when any of the following modem events occurs: incoming call, SMS received, loss of cellular service.

© Continental Automotive GmbH 2019
Confidential and Proprietary

> The **NADIF_modem_set_wakeup_reason** API would be invoked with the `wakeup_reasons` parameter set to a bit-wise OR of `NADIF_WU_INCOMING_VOICE_CALL`, `NADIF_WU_INCOMING_SMS`, and `NADIF_WU_LOSS_OF_SERVICE`:
>
> ```
> wakeup_reasons = NADIF_WU_INCOMING_VOICE_CALL | NADIF_WU_INCOMING_SMS | NADIF_WU_LOSS_OF_SERVICE;
> ```
>
> **Note: The requested `wakeup_reasons` value overwrites the current setting (not OR'ed with current setting).**

- Prepare external services for application processor suspend/low power mode. External peripherals such as GNSS receiver, USB, or WiFi probably should be turned off prior to suspending to conserve power while in low power mode.
- Enable any interrupts for waking up the application processor from an external source, such as an external power control module.
- Disable modem events reporting before suspend to avoid unsolicited indications from the NAD

> **Note**: The NADIF_set_event_report API should be invoked with the event_state field of the NADIF_EVENT_REPORT_REQ input parameter set to NADIF_EVENT_REPORT_DISABLE;

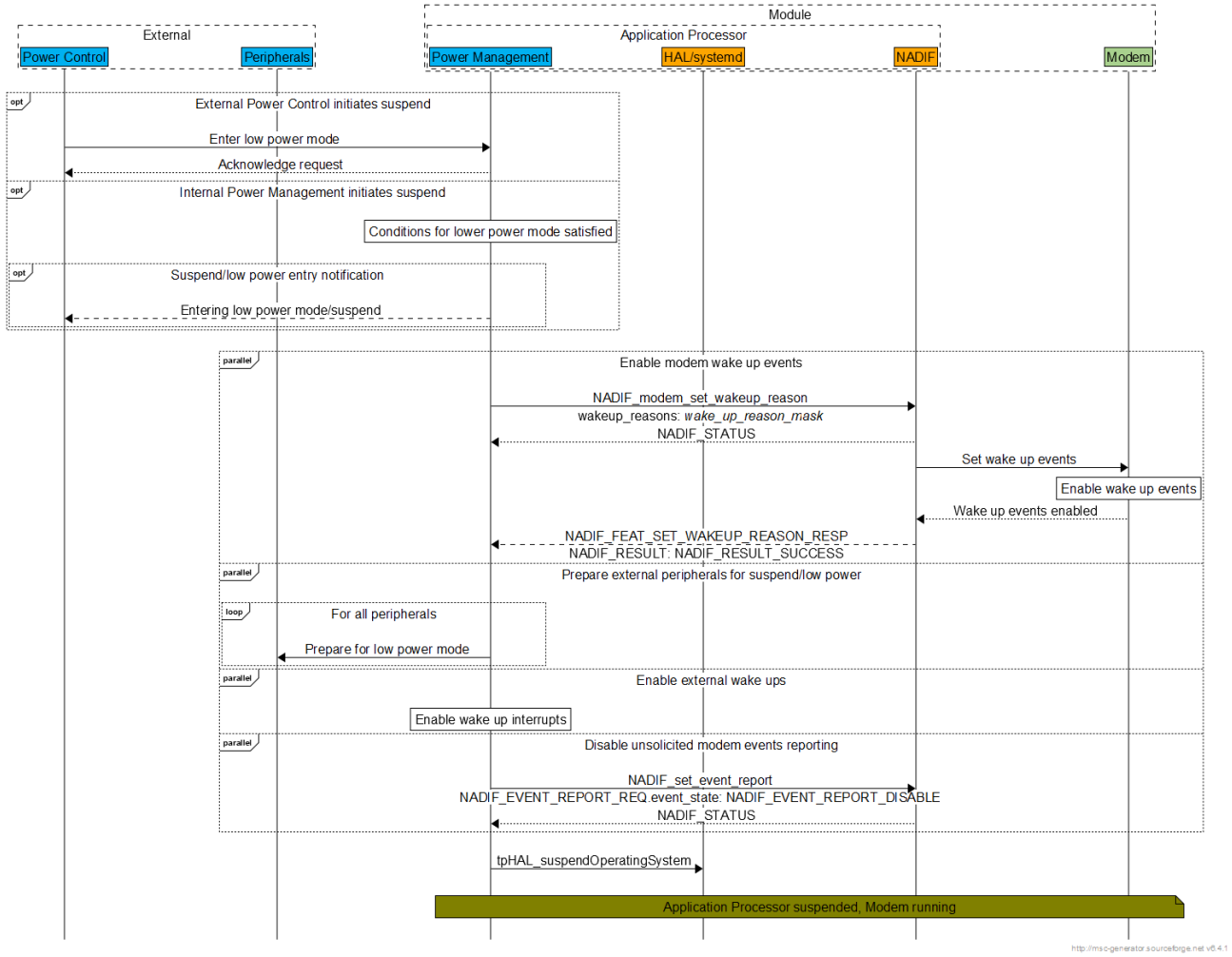
After preparing for operating system suspend, the **tpHAL_suspendOperatingSystem** API is invoked to suspend the application processor.

> **Notes:**
>
> 1) The application that invokes **tpHAL_suspendOperatingSystem** API must be a member of "power" group.
> 2) When the application processor resumes from being suspended, processing will continue with the next instruction/line of code after the call to **tpHAL_suspendOperatingSystem**. The application that invokes **tpHAL_suspendOperatingSystem** should be prepared to perform resume operations when continuing.

Module low power mode entry/application processor suspend sequence is illustrated in Figure 44.

© Continental Automotive GmbH 2019
Confidential and Proprietary

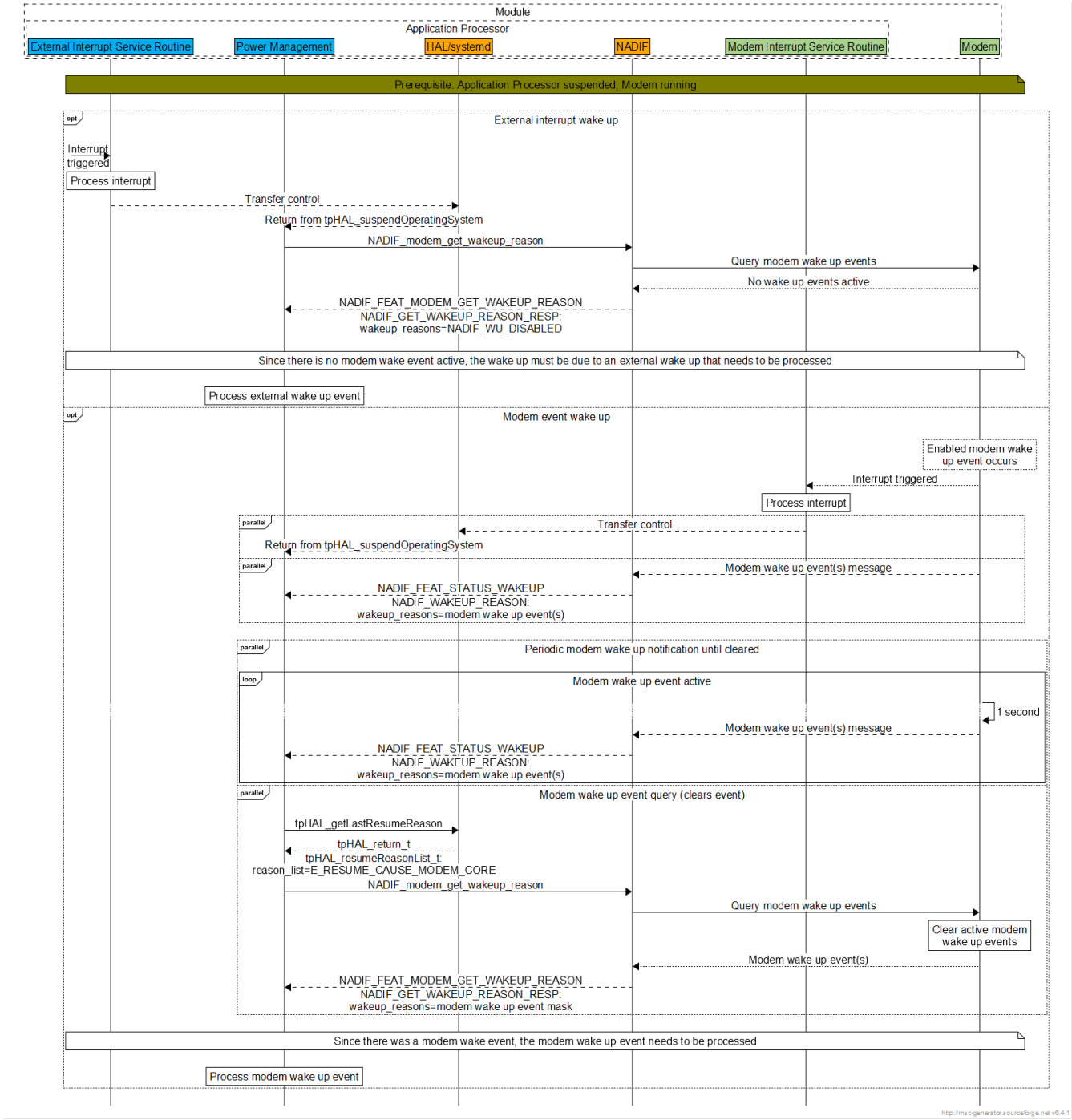
Figure 44: Suspend/low power mode entry sequence



Module resume/wake up from application processor suspend sequence is illustrated in Figure 55.

© Continental Automotive GmbH 2019
Confidential and Proprietary

Figure 55: Module resume/wake up sequence



After resuming operation from suspend/low power mode, the following actions may be taken as appropriate based on wake up event:

- Determine wake up source. To determine that the wake up/resume was caused by a modem wake up event the `tpHAL_getLastResumeReason` API should be used with the

© Continental Automotive GmbH 2019
Confidential and Proprietary

E_RESUME_CAUSE_MODEM_CORE present on the output parameter of resume reasons list. If processor wake up/resume was caused by a modem wake up event, the **NADIF_get_wakeup_reason** API must be invoked to stop unsolicited **NADIF_FEAT_STATUS_WAKEUP** notifications and clear the modem wake up event

- Process the wake up event(s) – e.g., answer incoming call, read received SMS, handle external power mode change requestRestart/reenable external services, especially if it is determined that the application processor will remain awake (i.e., will not be suspended) after finished processing the wake up event(s).
- If it is determined that the application processor will remain awake (i.e., will not be suspended) after finished processing the wake up event(s), disable modem events that will trigger an application processor wake up (resume). Otherwise, such events will trigger wake up event notifications when awake and those events will still need to be cleared in the modem. The NADIF API **NADIF_modem_set_wakeup_reason** is used for this purpose. The API is invoked with the wakeup_reasons input parameter set to a value of `NADIF_WU_DISABLE`.

Note: Low Power Mode duration timing is out of the scope of this document. Such timing could be implemented via an external timer that is tied to an external wakeup. When the wakeup occurs, a timer expiry notification is provided to Power Management, which will then proceed with a Module Power Down. Such timing could also be implemented via an internal timer such as a Linux system timer. Power Management is aware of the internal timer and can initiate a Module Power Down upon timer expiration.

3.5.3.3 Testing/Error Reporting

1) **NADIF_FEAT_STATUS_WAKEUP** notifications continue to be received periodically after requesting modem wake up events via **NADIF_modem_get_wakeup_reason**

   It is possible for multiple modem wake up events to occur within a short period of time or for the same event to occur multiple times in a short period. As an example, the modem could receive a SMS message and then get an incoming phone call shortly after, triggering both a `NADIF_WU_INCOMING_SMS` and `NADIF_WU_INCOMING_VOICE_CALL` wake up event notification. As another example, the modem could receive more than one SMS message in succession, with each message triggering a separate `NADIF_WU_INCOMING_SMS` wake up event notification. As such, it is possible for another modem wake up event to be triggered just after the modem wake up event/reason request is made.

   If **NADIF_FEAT_STATUS_WAKEUP** notifications are received after clearing active modem wake up events via **NADIF_modem_get_wakeup_reason** API, **NADIF_modem_get_wakeup_reason** API should be invoked again to process the new wake up events. If the application processor is not going to be suspended (i.e., will remain awake after processing wake up events), then all modem wake up event notifications should be disabled by invoking **NADIF_modem_set_wakeup_reason** API with the `wakeup_reasons` input parameter set to a value of `NADIF_WU_DISABLED`.

2) **NADIF_FEAT_STATUS_WAKEUP** notifications are being received periodically after wake up from an external wake up source

   A modem wake up event occurred at about the same time as an external wake up event. The modem wake up event needs to be read/cleared and processed. Use **NADIF_modem_get_wakeup_reason** API to read the modem wake up event(s) that occurred and clear the active modem wake ups.

3) A wake up occurred, but **NADIF_modem_get_wakeup_reason** API returns a value of `NADIF_WU_DISABLED` when invoked

   The first and most likely possible cause for this issue is the wake up was triggered by an external source and not the modem. In this case no modem wake up events are active, so the external wake up event needs to be processed.

© Continental Automotive GmbH 2019
Confidential and Proprietary

The second possible cause for this issue is some other NADIF client invoked the **NADIF_modem_get_wakeup_reason** API before the power manager application. Thus, when the power manager application queries the active modem wake up events, they have already been cleared. To overcome this issue, ensure that only one NADIF client handles processing modem wake up events (setting wake up event mask prior to suspend, requesting wake up events after resume, disabling wake up events when awake, etc.). This common application can then disseminate wake up event information to other applications as needed.

3.6 Packet Data during Low Power Mode

3.6.1 Overview

3.6.1.1 Platform Content Overview

The Telematics Platform provides support to maintain a packet data session while module is in Low Power Mode. The following packet data modem wake up events are provided by the platform:

- `NADIF_WU_PKT_DATA_STATE_ACTIVE`: When enabled, the modem will wake up the application processor and set `NADIF_WU_PKT_DATA_STATE_ACTIVE` wake up event active when the packet data session status transitions from dormant to active. This transition will occur when there is incoming packet traffic received by the modem.
- `NADIF_WU_PKT_DATA_STATE_DORMANT`: When enabled, the modem will wake up the application processor and set `NADIF_WU_PKT_DATA_STATE_DORMANT` wake up event active when the packet data session status transitions from active to dormant. This transition may occur when the data inactivity timer times out and the application processor was suspended when the packet data session status was still active.

3.6.1.2 Product Expectations

1) Set up packet data session prior to entering module Low Power Mode (e.g., prior to suspending application processor).
2) Enable `NADIF_WU_PKT_DATA_STATE_ACTIVE` modem wake up event as part of low power mode preparation. Optionally enable `NADIF_WU_PKT_DATA_STATE_DORMANT` modem wake up event as well.
3) Follow Product Expectations for Low Power Mode.
4) Wake up periodically (e.g., once an hour, once every 4 hours) to maintain packet data connection with the cellular network.
5) (recommended) Wait for packet data session to transition to dormant state prior to entering low power mode. In other words, it is best that the packet data session is already dormant when suspending the application processor.
6) Process received packet data after a wake up and the modem wake up event/reason includes `NADIF_WU_PKT_DATA_STATE_ACTIVE`.

3.6.1.3 Extending Platform Functionality

Not Applicable

3.6.2 Setup

3.6.2.1 Tools/Equipment Needed

Not Applicable

© Continental Automotive GmbH 2019
Confidential and Proprietary

3.6.2.2 Setup Instructions

NADIF APIs should be used for modem wakeup event access. A NADIF client (see TP_F101U-00036) is required for accessing NADIF APIs and receiving NADIF event information. Only one NADIF client should control and process all modem wakeup events, including packet data session related events. This single client can disseminate wakeup information to other application components as necessary.

A packet data session must be established prior to application processor suspend. Recommendation is to let the packet data session go dormant prior to application processor suspend.

3.6.2.3 Setup Troubleshooting

Not Applicable

3.6.3 Usage

3.6.3.1 Basic Instructions

General module low power mode with packet data session sequence:

1) Set up cellular packet data session and connect to server.
2) (recommended) Wait for packet data session state transition to dormant state.
3) Enable `NADIF_WU_PKT_DATA_STATE_ACTIVE` and `NADIF_WU_PKT_DATA_STATE_DORMANT` (if desired) modem wake up events in addition to other modem wake up events for application processor suspend via NADIF.
4) Start a “packet data session maintenance” timer as a wake up source to maintain packet data session (recommendation: minimum 1 hour, maximum 8 hours).
5) Perform other preparations for low power mode/suspend and request systemd suspend.
6) Upon resume, if wake up source is “packet data session maintenance” timer expiration:
   a. Perform a ping or some other “send” action for packet data session. This will reset the cellular network packet data dormancy timer to prevent tear down of the packet data connection by the cellular service provider.
   b. Wait for packet data session state transition to dormant state.
   c. Restart “packet data session maintenance” timer.
   d. Request systemd suspend to return to Low Power Mode.
7) Upon resume, if wake up source is NADIF_WU_PKT_DATA_STATE_ACTIVE:
   a. Process data received from packet data far end connection.
   b. Wait for packet data session state transition to dormant state after processing packet data.
   c. Restart “packet data session maintenance” timer.
   d. Request systemd suspend to return to Low Power Mode.

3.6.3.2 Detailed Instructions

NADIF APIs to be used for packet data session establishment and session status updates.

API/Type Description	Doxygen Path

NADIF API to establish a packet data session	`Modules`
	`|-> Modem`
	`|-> NADIF`
	`|-> Data Services`
	`|-> NADIF_data_start`

© Continental Automotive GmbH 2019
Confidential and Proprietary

NADIF API to obtain packet data session status	Modules |-> Modem |-> NADIF |-> Data Services |-> **NADIF_data_get_status**
Unsolicited NADIF packet data session status change notification	Modules |-> Modem |-> NADIF |-> Data Services/Notifications |-> **NADIF_FEAT_STATUS_DATA**
API to set modem events that will trigger a wakeup event notification	Modules |-> Modem |-> NADIF |-> Modem Status |-> **NADIF_modem_set_wakeup_reason**
Modem set wakeup events response notification	Modules |-> Modem |-> NADIF |-> NADIF Common Types |-> **NADIF_RESULT**
Unsolicited modem wakeup event notification	Modules |-> Modem |-> NADIF |-> Modem Status/Notifications |-> **NADIF_FEAT_STATUS_WAKEUP**
API to query modem event(s) that caused a processor wake up	Modules |-> Modem |-> NADIF |-> Modem Status |-> **NADIF_get_wakeup_reason**
Modem get wakeup reason response notification	Modules |-> Modem |-> NADIF |-> Modem Status

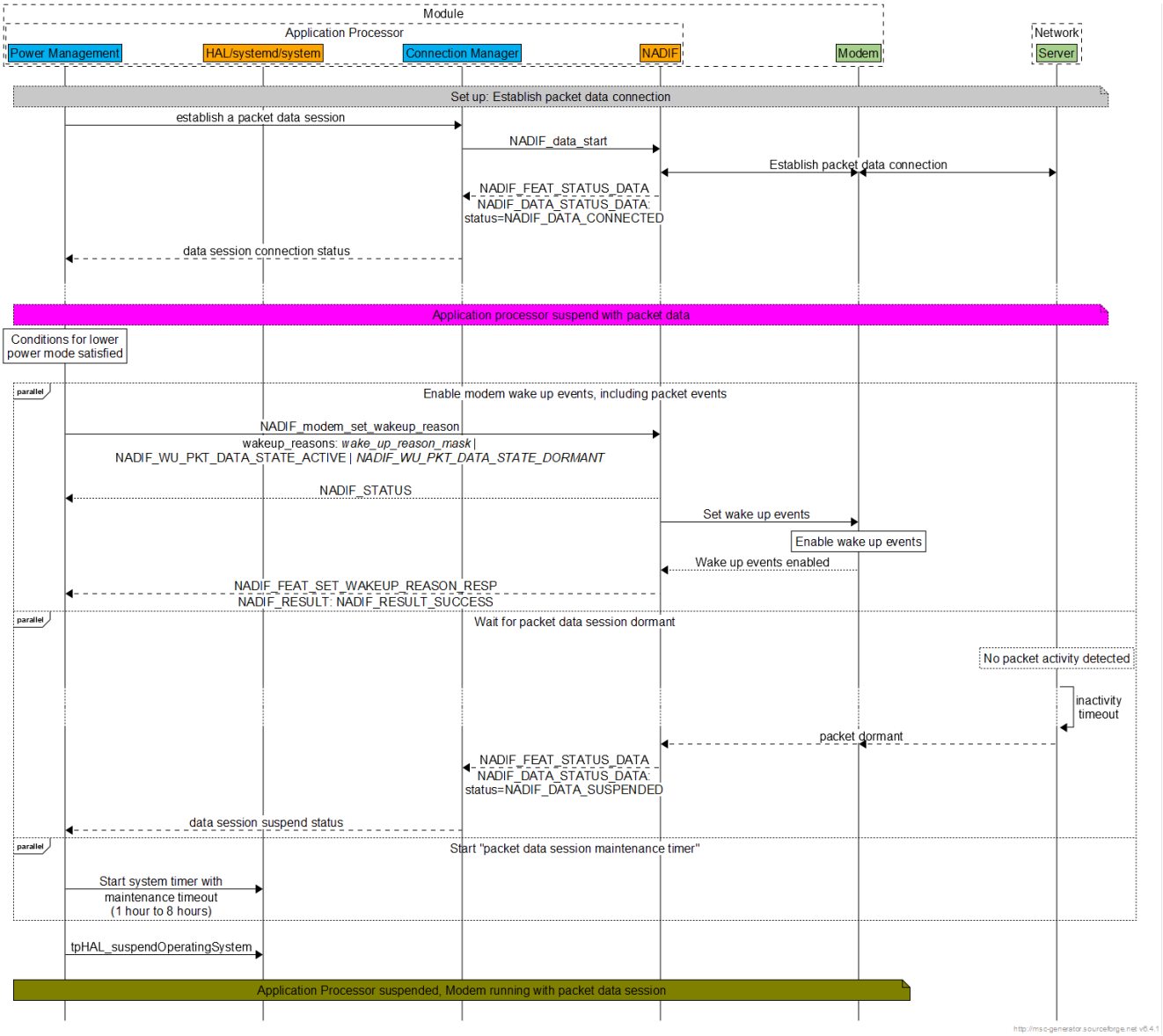
© Continental Automotive GmbH 2019
Confidential and Proprietary

	|-> **NADIF_GET_WAKEUP_REASON_RESP**
API to request operating system suspend	Modules |-> Hardware Abstraction Layer (HAL) |-> HAL Library |-> System Interface |-> **tpHAL_suspendOperatingSystem**

Refer to Low Power Mode – suspend/resume for details on application processor suspend/resume operation.

Module low power mode entry/application processor suspend with packet data session is illustrated in Figure 66.

Figure 66: Suspend/low power mode entry with packet data sequence



© Continental Automotive GmbH 2019
Confidential and Proprietary

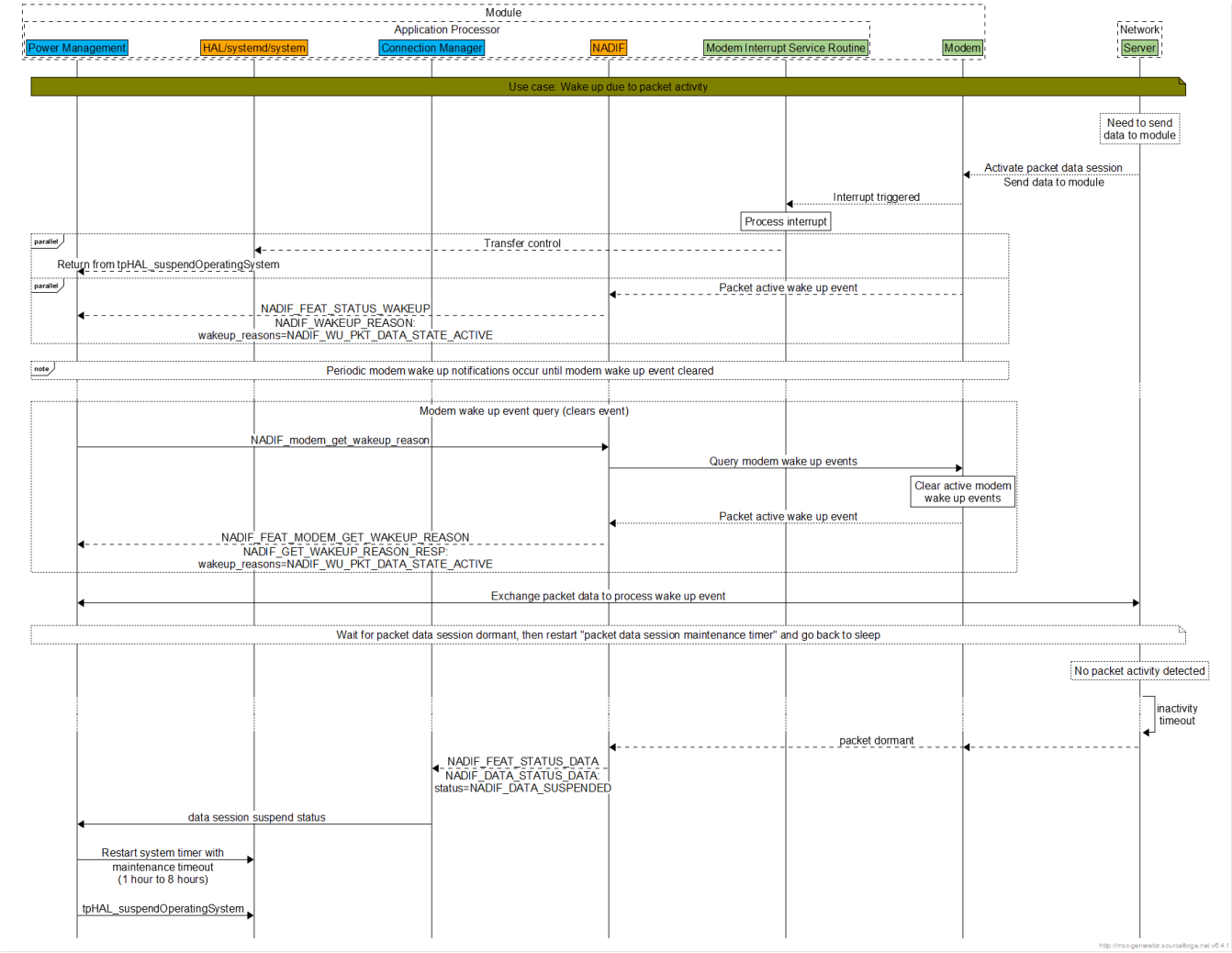
Note: It is recommend that Power Management waits for packet data dormant state before suspending the application processor. Most cellular networks have a relatively short inactivity/dormant mode timeout. This approach simplifies wake up logic/processing and overall power consumption is comparable. If the Power Management always waits for packet data dormant, then enabling `NADIF_WU_PKT_DATA_STATE_DORMANT` event is not necessary.

Module resume/wake up from application processor suspend due to packet data activity is illustrated in Figure 77. Module resume/wake up from application processor suspend due to "packet data session maintenance" timer expiration is illustrated in Figure 88. Note that these sequences assume that a packet data event is the only wake up event/reason and the application processor is suspended once the packet data event is processed.

Note: Low Power Mode and Packet Data session duration timing is out of the scope of this document. Such timing could be implemented via an external timer that is tied to an external wakeup. When the wakeup occurs, a timer expiry notification is provided to Power Management, which will then proceed with terminating the packet data session and/or Module Power Down as appropriate. Such timing could also be implemented via an internal timer such as a Linux system timer. Power Management is aware of the internal timer and can initiate termination of the packet data session and/or Module Power Down as appropriate upon timer expiration.

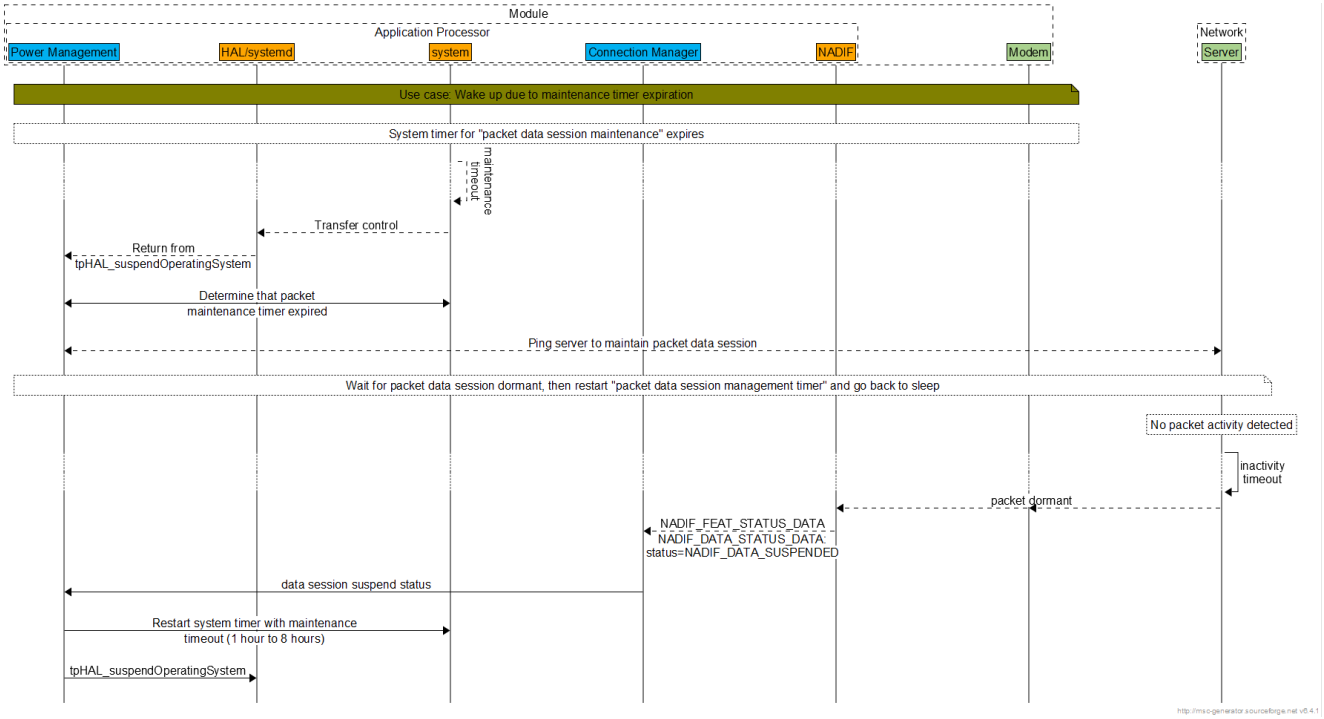
© Continental Automotive GmbH 2019
Confidential and Proprietary

Figure 77: Module resume/wake up due to packet data active sequence



© Continental Automotive GmbH 2019
Confidential and Proprietary

Figure 88: Module resume/wake up due to packet maintenance timer expiration sequence



3.6.3.3 Testing/Error Reporting

Not Applicable

3.7 Power Related GPIO

3.7.1 Overview

3.7.1.1 Platform Content Overview

The Telematics Platform completely or partially controls some power related GPIOs. Other GPIOs are available for general use, including power related operations. GPIO access and configuration is provided via the Linux sysfs component.

Power related GPIO completely controlled by Telematics Platform: RESET_N, PM_KPD_PWR_N, VUC2MDM_HOLDUP, NAD2PS_HOLDUP, NAD_READY, NAD_LPM

Power related GPIO partially controlled by Telematics Platform: BOOT_CFG1, BOOT_CFG2, BT_UART_TX, GPIO_F14, GPIO_F18, FORCED_USB_BOOT

Power related GPIO unused by Telematics Platform: NAD2TCU_WAKE

3.7.1.2 Product Expectations

1) Do not override or interfere with power related GPIO completely controlled by Telematics Platform: RESET_N, PM_KPD_PWR_N, VUC2MDM_HOLDUP, NAD2PS_HOLDUP, NAD_READY, NAD_LPM
2) Avoid using the boot mode configuration pins BOOT_CFG1, BOOT_CFG2, BT_UART_TX, GPIO_F14, GPIO_F18, and FORCED_USB_BOOT. If the pins are connected as an output on system hardware, the

© Continental Automotive GmbH 2019
Confidential and Proprietary

pins may be configured as an output via sysfs after power up initialization is complete and driven accordingly.

3) Use sysfs for configuring additional GPIO used for power management operations (e.g., external wake-up from suspend, external request to stay awake).
4) Use sysfs for configuring General Purpose inputs as interrupt sources prior to initiating module suspend for Low Power Mode (e.g., external wake-up from suspend).

3.7.1.3 Extending Platform Functionality

sysfs can be used to configure additional GPIO inputs and/or outputs to be used for power management operation as indicated on system schematic.

3.7.2 Setup

3.7.2.1 Tools/Equipment Needed

Not Applicable

3.7.2.2 Setup Instructions

sysfs should be used for configuring and controlling power related GPIOs that are not completely controlled by Telematics Platform. Each GPIO can be configured as an input or output via sysfs. Output GPIOs can be driven high or low via sysfs and the value of input GPIOs can be read via sysfs. Additionally, sysfs can be used to configure input GPIOs as an interrupt source during module suspend (edge triggered).

3.7.2.3 Setup Troubleshooting

Not Applicable

3.7.3 Usage

3.7.3.1 Basic Instructions

Power related GPIOs completely controlled by Telematics Platform: Do not use.

- RESET_N: Input to NAD module, active low.
- PM_KPD_PWR_N: Input to NAD module, active low.
- VUC2MDM_HOLDUP: Input to NAD module hardware, active high.
- NAD2PS_HOLDUP: Output from NAD module, active high.
- NAD_READY: Output from NAD module, reserved for platform use.
- NAD_LPM: Output from NAD module, used to indicate suspended/low power mode.

Power related GPIOs partially controlled by Telematics Platform: Limited use

- BOOT_CFG1, BOOT_CFG2, BT_UART_TX, GPIO_F14, GPIO_F18, FORCED_USB_BOOT: Inputs to the NAD module during module power up. Used during module power up to determine module boot mode. These lines can be used/configured as outputs only after power up initialization is complete, but cannot be used as inputs (recommend that they are unused if possible). The lines cannot be pulled low or pulled high externally, as this could affect module boot up when a module power cycle occurs.

Power related GPIO unused by Telematics Platform:

- NAD2TCU_WAKE: This pin is free and can be used by customer power management to assert Vehicle wake up.

Configuring a GPIO for Power Management:

© Continental Automotive GmbH 2019
Confidential and Proprietary

1) Locate GPIO on system schematic to determine pin/pad name/designator.
2) Locate Pad designator (e.g., E16) or LGA Pad Name (e.g., GPIO_E16) on "SA515M Hardware Functions and GPIO Assignment" table in 5G NAD Module device specification.
3) Find GPIO pin number associated with Pad designator and/or LGA Pad Name in the "SA515 GPIO Pin" column in the table. Pin number is the number following "GPIO_" (e.g., pin number for Pad E16 is 104).
4) Use GPIO pin number to find associated sysfs folder for GPIO access/control (e.g., `/sys/class/gpio/gpio<id>`; Pad E16 sysfs folder would be `/sys/class/gpio/gpio104`).
5) Access GPIO file via standard POSIX `open`/`read`/`write`/`close` APIs.

3.7.3.2 Detailed Instructions

3.7.3.2.1 Telematics Platform power management related GPIO – full control

The following power management related GPIO is under full control of Telematics Platform:

- RESET_N: Input to NAD module, active low. Used to reset the module when module is unresponsive. State/value can be read, but reset pulse (i.e., line pulled low) will be processed directly by systemd.
- PM_KPD_PWR_N: Input to NAD module, active low. Used for module power up and module power down. State/value can be read, but power pulse (i.e., line pulled low) will be processed directly by the Power Management IC (power up) or systemd (power down).
- VUC2MDM_HOLDUP: Input to NAD module hardware, active high. Used for ensuring uniform and consistent power is provided to the module. State/value can be read, not processed within software.
- NAD2PS_HOLDUP: Output from NAD module, active high. Used for ensuring uniform and consistent power is provided to the module. Line is driven active during module boot up/power up and is driven inactive during module power down by Linux kernel. Changing the state of this output during module operation could cause an unintended sudden module power down.
- NAD_READY, NAD_LPM: Outputs from NAD module. Used to indicate NAD module operating status as shown in Table 11**Error! Reference source not found.**.

Table 11: Module status indication

GPIO	GPIO state	Operating state description
NAD_READY	0 (low)	*Reserved*
NAD_READY	1 (high)	*Reserved*
NAD_LPM	0 (low)	NAD module is running (i.e., not suspended) when NAD2PS_HOLDUP is also asserted. Set by Telematics Platform during boot up sequence. Also set by Linux kernel when resuming from suspend due to a wake-up interrupt (e.g., prior to return from `tpHAL_suspendOperatingSystem` API call).
NAD_LPM	1 (high)	NAD module is in Low Power Mode (application processor suspended) state. Set by Linux kernel just before suspending (e.g., due to `tpHAL_suspendOperatingSystem` API invocation).

3.7.3.2.2 Telematics Platform power management related GPIO – partial control

HOWTO Use boot config pins as output (not recommended)

The boot config pins BOOT_CFG1, BOOT_CFG2, BT_UART_TX, GPIO_F14, GPIO_F18, and FORCED_USB_BOOT are used as inputs to the NAD module during module power up to determine the

© Continental Automotive GmbH 2019
Confidential and Proprietary

module boot mode. As such, these pins should not be configured as a General Purpose input and should not be pulled high or pulled low external to the module. Recommendation is the pins are not connected in hardware and thus not used.

If a boot config pin is connected as an output in hardware, the corresponding pin can be configured as an output and set high or low as needed. As an example, if the Bluetooth UART is used, then the BT_UART_TX GPIO should be configured as an output during UART initialization and used for the UART transmit line.

3.7.3.2.3 General GPIO configuration and access

HOWTO Configure a GPIO input and read its value

1. Export GPIO if not already exported
2. Set GPIO direction as “in”
3. Read GPIO value
4. Unexport GPIO when finished

HOWTO Configure a GPIO output and set its state

1. Export GPIO if not already exported
2. Set GPIO direction as “out”
3. Write GPIO value
4. Unexport GPIO when finished

HOWTO Configure a GPIO input as an interrupt source prior to suspend

1. Find a GPIO that can be used as a wakeable interrupt (see “Wakeable Interrupts” column on “SA515M Hardware Functions and GPIO Assignment” table in 5G NAD Module device specification). For example, Pad E16 (GPIO_104) can be configured as a wakeable interrupt, but Pad E17 (GPIO_105) cannot.
2. Export GPIO if not already exported
3. Confirm GPIO configured as an input or configure as input
4. Set GPIO interrupt edge trigger(s)
5. Unexport GPIO when finished

HOWTO Determine if a GPIO interrupt caused module wake up/resume from suspend

All GPIOs trigger a single system interrupt. The interrupt number for the GPIO interrupt is defined in **`/var/resume_reasons/msm_gpio_cas_irq_num`** file. When the system resumes from suspend, the interrupt numbers for all interrupts that triggered the wake up event are captured in **`/sys/kernel/wakeup_reasons/last_resume_reason`**. So, after context is returned from `tpHAL_suspendOperatingSystem` API, if the GPIO interrupt number obtained from **`/var/resume_reasons/msm_gpio_cas_irq_num`** matches any of the interrupt numbers listed in **`/sys/kernel/wakeup_reasons/last_resume_reason`**, then a GPIO interrupt occurred.

1. *Sequence start: return from tpHAL_suspendOperatingSystem API call.*
2. Determine/obtain interrupt number for GPIO interrupt.
3. Iterate through list of interrupt numbers for interrupts that triggered the wake up event to match GPIO interrupt number.
4. If GPIO interrupt number matches an interrupt number in the wake up event interrupt list, a GPIO interrupt occurred.
5. Take additional steps as needed to determine which GPIO caused the wake up if multiple GPIO interrupts are supported. For example, the value of each GPIO could be read to determine which GPIO is active. Then, the GPIO can be deasserted.
6. (optional) Remove GPIO interrupt edge trigger(s)

© Continental Automotive GmbH 2019
Confidential and Proprietary

Example Code

Export GPIO if not already exported

```
#define MAX_GPIO_PATH_SIZE 50

gint fd =  -1;
gint ret = 0;
gchar gpio_path[MAX_GPIO_PATH_SIZE];
gboolean skip_export = FALSE;
struct stat sbuf;

g_snprintf(gpio_path, MAX_GPIO_PATH_SIZE - 1, "/sys/class/gpio/gpio%d",
<gpio number>);
gpio_path[MAX_GPIO_PATH_SIZE - 1] = '\0';
if(stat(gpio_path, &sbuf) != -1)
{
    if((sbuf.st_mode & S_IFMT) == S_IFDIR)
    {
        skip_export =- TRUE;
    }
}
if(skip_export != TRUE)
{
    fd = open("/sys/class/gpio/export", O_WRONLY);
    if(fd < 0)
    {
        // failed to open file, report error and stop processing
        ...
    }
    ret = write(fd, "<gpio number>", <gpio number string size>);
    if(ret != <gpio number string size>)
    {
        // file write failed, report error, close file, stop processing
        close(fd);
        ...
    }
    close(fd);
}
```

Set GPIO direction as "in"

```
gint fd = -1;
gint ret = 0;

fd = open("/sys/class/gpio/gpio<gpio number>/direction", O_WRONLY);
if(fd < 0)
{
    // failed to open file, report error and stop processing
    ...
}
ret = write(fd, "in", 2);
if(ret != 2)
{
    // file write failed, report error, close file, stop processing
```


© Continental Automotive GmbH 2019
Confidential and Proprietary

```
        close(fd);
        ...
    }
    close(fd);
```

Set GPIO direction as “out”

```
    gint fd = -1;
    gint ret = 0;

    fd = open(“/sys/class/gpio/gpio<gpio number>/direction”, O_WRONLY);
    if(fd < 0)
    {
        // failed to open file, report error and stop processing
        ...
    }
    ret = write(fd, “out”, 3);
    if(ret != 3)
    {
        // file write failed, report error, close file, stop processing
        close(fd);
        ...
    }
    close(fd);
```

Read GPIO value

```
    #define MAX_VALUE_SIZE 10

    gint fd = -1;
    gchar gpio_buf[MAX_VALUE_SIZE];
    gint ret = 0;

    fd = open(“/sys/class/gpio/gpio<gpio_number>/value”, O_RDONLY);
    if(fd < 0)
    {
        // failed to open file, report error and stop processing
        ...
    }
    ret = read(fd, gpio_buf, MAX_VALUE_SIZE);
    if(ret <= 0)
    {
        // file read failed, report error, close file, stop processing
        close(fd);
        ...
    }
    gpio_buf[1] = ‘\0’;
    close(fd);
```

Write GPIO value

```
    gint fd = -1;
    gint ret = 0;

    fd = open(“/sys/class/gpio/gpio<gpio_number>/value”, O_WRONLY);
    if(fd < 0)
```


© Continental Automotive GmbH 2019
Confidential and Proprietary

```
{
    // failed to open file, report error and stop processing
}
// set GPIO to low (0) value
ret = write(fd, "0", 1);
if(ret != 1)
{
    // file write failed, report error, close file, stop processing
    close(fd);
    ...
}
// set GPIO to high (1) value
ret = write(fd, "1", 1);
if(ret != 1)
{
    // file write failed, report error, close file, stop processing
    close(fd);
    ...
}
close(fd);
```

Confirm GPIO configured as an input

```
#define MAX_DIR_SIZE 10
#define GPIO_DIR_IN "in"

gint fd = -1;
gchar gpio_buf[MAX_DIR_SIZE];
gint ret = 0;

fd = open("/sys/class/gpio/gpio<gpio number>/direction", O_RDONLY);
if(fd < 0)
{
    // failed to open file, report error and stop processing
    ...
}
ret = read(fd, gpio_buf, MAX_DIR_SIZE);
if(ret <= 0)
{
    // file read failed, report error, close file, stop processing
    close(fd);
    ...
}
if(g_ascii_strncasecmp(GPIO_DIR_IN, direction, sizeof(GPIO_DIR_IN)) ==
0)
{
    // GPIO configured as an input, indicate positive response
}
else
{
    // GPIO not configured as an input, indicate negative response
}
```

Set GPIO interrupt edge trigger(s)

© Continental Automotive GmbH 2019
Confidential and Proprietary

```
gint fd = -1;
gint ret = 0;

fd = open("/sys/class/gpio/gpio<gpio number>/edge", O_WRONLY);
if(fd < 0)
{
    // failed to open file, report error and stop processing
    ...
}

// set interrupt on rising edge
ret = write(fd, "rising", 6);
if(ret != 6)
{
    // file write failed, report error, close file, stop processing
    close(fd);
    ...
}

// set interrupt on falling edge
ret = write(fd, "falling", 7);
if(ret != 7)
{
    // file write failed, report error, close file, stop processing
    close(fd);
    ...
}

// set interrupt on both edges
ret = write(fd, "both", 4);
if(ret != 4)
{
    // file write failed, report error, close file, stop processing
    close(fd);
    ...
}

close(fd);
```

Unexport GPIO

```
gint fd = -1;
gint ret = 0;

fd = open("/sys/class/gpio/unexport", O_WRONLY);
if(fd < 0)
{
    // failed to open file, report error and stop processing
    ...
}
ret = write(fd, "<gpio number>", <gpio number string size>);
if(ret != <gpio number string size>)
{
    // file write failed, report error, close file, stop processing
```


© Continental Automotive GmbH 2019
Confidential and Proprietary

```
        close(fd);
        ...
    }
    close(fd);
```

Determine GPIO interrupt number

```
    #define MAX_INTNUM_SIZE 10

    FILE *fp = NULL;
    gchar gpio_buf[MAX_INTNUM_SIZE];
    gchar *endptr = NULL;
    gint GPIO_num;

    fp = fopen("/var/resume_reasons/msm_gpio_cas_irq_num", "r");
    if(fp == NULL)
    {
        // failed to open file, report error and stop processing
        ...
    }
    if(fgets(gpio_buf, sizeof(gpio_buf), fp) != NULL)
    {
        GPIO_num = g_ascii_strtoll(gpio_buf, endptr, 0);
        if(gpio_buf == endptr)
        {
            // conversion to integer failed, report error and stop
            // processing
            fclose(fp);
            ...
        }
    }
    else
    {
        // file read failed, report error, close file, stop processing
        fclose(fp);
        ...
    }
    fclose(fp);
```

Determine if GPIO interrupt caused wake up

```
    #define MAX_LINE_SIZE 25
    extern gint GPIO_num;

    FILE *fp = NULL;
    gchar line_buf[MAX_LINE_SIZE];
    gchar *endptr;
    gint irq_num;
    Boolean gpio_found = FALSE;

    fp = fopen("/sys/kernel/wakeup_reasons/last_resume_reason", "r");
    if(fp == NULL)
    {
        // failed to open file, report error and stop processing
        ...
```


© Continental Automotive GmbH 2019
Confidential and Proprietary

```
}

while(!gpio_found && fgets(line_buf, sizeof(line_buf), fp) != NULL)
{
    endptr = NULL;
    irq_num = g_ascii_strtoll(line_buf, endptr, 0);
    if(line_buf != endptr)
    {
        if(irq_num == GPIO_num)
        {
            // match found
            gpio_found = TRUE;
        }
    }
}

fclose(fp);
// if gpio_found is TRUE then a GPIO interrupt triggered the wake up
// if gpio_found is FALSE then something other than a GPIO interrupt
//  triggered the wake up
```

Remove GPIO interrupt edge trigger(s)

```
gint fd = -1;
gint ret = 0;

fd = open("/sys/class/gpio/gpio<gpio number>/edge", O_WRONLY);
if(fd < 0)
{
    // failed to open file, report error and stop processing
    ...
}

// set interrupt to none
ret = write(fd, "none", 4);
if(ret != 4)
{
    // file write failed, report error, close file, stop processing
    close(fd);
    ...
}
Close(fd);
```

Example putting it all together

The example code file below is a simple example of using 2 GPIOs for power related operations. One GPIO is used as an input for a keep awake request. When the “keep awake” GPIO is low (i.e., not asserted) then the module is allowed to go to sleep when all other sleep conditions are met. When the “keep awake” GPIO is high (i.e., asserted) then the module is not allowed to go to sleep and must wait for the “keep awake” GPIO to go low. The other GPIO is used as an external wake up source. When the “external wake up” GPIO transitions from low to high while the module is asleep/suspended, the module will wake up.

The example code includes an initialization API that could be called during power manager application start up, an API to request module suspend when allowed by “keep awake” GPIO, and a clean up API that could be called during application power down preparation.

© Continental Automotive GmbH 2019
Confidential and Proprietary

GPIO_IRQ_sample.c

GPIO_IRQ_sample.h

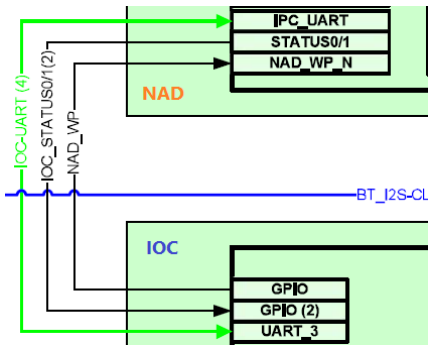
3.7.3.3 Testing/Error Reporting

Not Applicable

4 Watchdog & Error Handling

For the proper boot ups, thanks to the secure boot mechanism, the NAND corruption, manipulation or damaged image will be detected by checking the hash of the images to be loaded. RAM Memory problems will occur during the execution of loaded images and will also stop the system startup. In every startup in worst-cases the NAD application will not be running and as consequence no heartbeat can be sent from NAD to IOC via UART. This is the point where the IOC can detect the Startup problem on NAD.

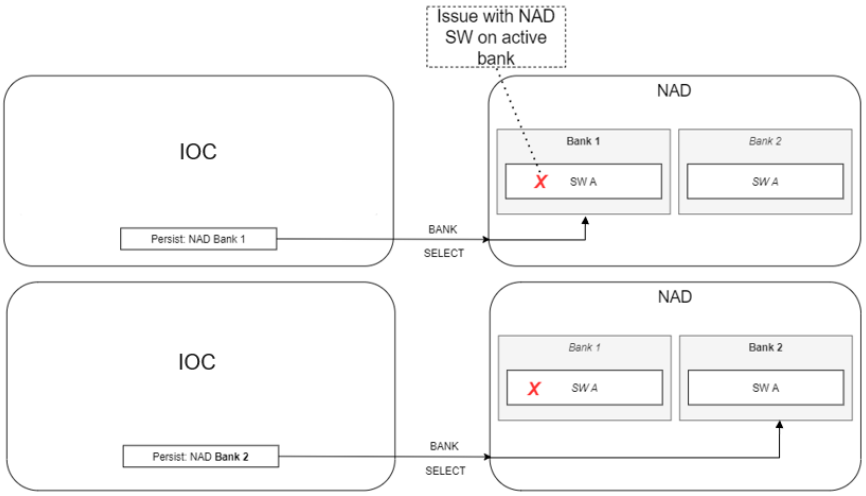
Figure 99: Module maintenance pin blocks



For the cases where the NAD has an issue with the active bank during boot up, it is possible to select the remaining bank as active and hence have the possibility to recover and boot up.

Figure 1010: Concept Summary: Changing active bank in case NAD cannot be booted up

© Continental Automotive GmbH 2019
Confidential and Proprietary

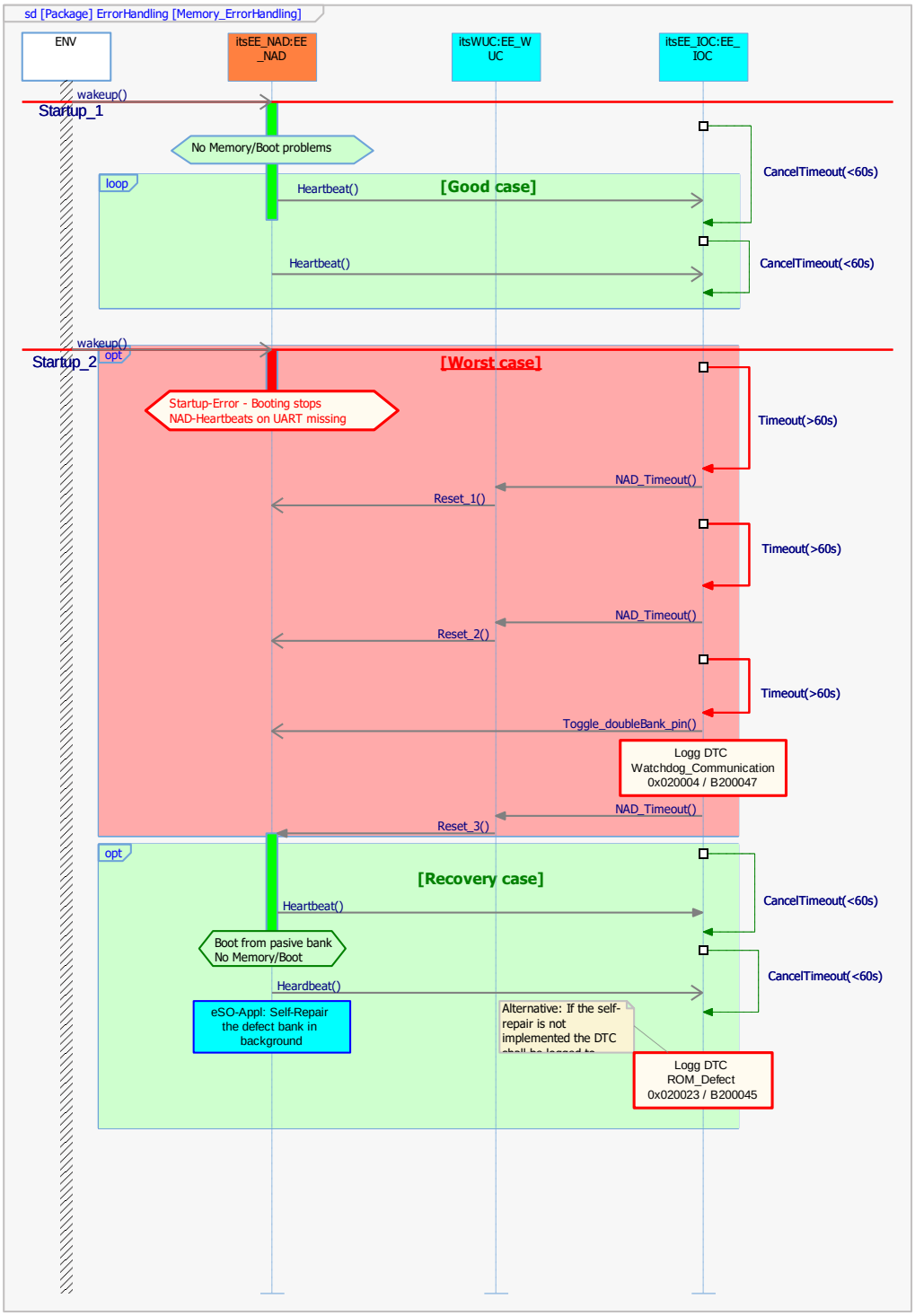


IOC shall initiate a restart of NAD via WUC when the NAD is not sending any heartbeat within expected time after boot up. This time duration depends on the IPC communication services of product team hence shall be measured and set accordingly.

The IOC shall continue resetting the NAD and If the NAD is not able to complete boot up three times in consequence, the IOC shall log the DTC “0x020004” (Watchdog_Communication) and then switch the NAD bank on pin VUC2NAD_BANK_SELECT to force the NAD to boot from the other bank. If the problem is in the active bank, the system shall be able to start and run from the passive bank (see recovery case below in the Figure 1111). In the recovery case eSolution application shall repair the damaged bank by overwriting it with the content of current bank. If the self-repaire is not done, at least the DTC 0x020023 (ROM_Defect) shall be logged. These steps are illustrated in Figure 1212.

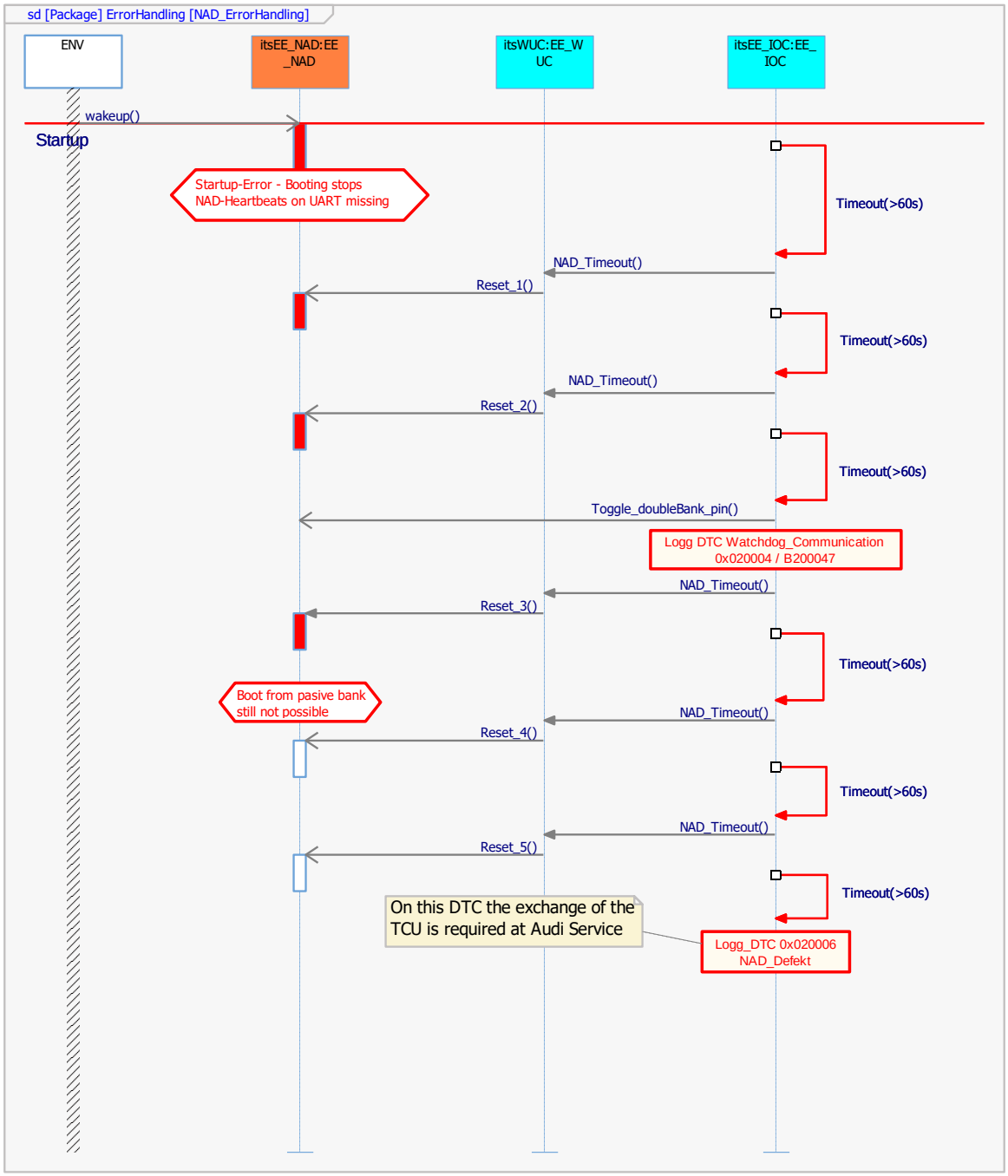
Figure 1111: Recovery from the passive bank

© Continental Automotive GmbH 2019
Confidential and Proprietary



© Continental Automotive GmbH 2019
Confidential and Proprietary

Figure 1212: Logging DTC in case NAD cannot be booted up from both banks



5 Known Issues

Not Applicable

© Continental Automotive GmbH 2019
Confidential and Proprietary

6 FAQs

Not Applicable

7 Troubleshooting/Tips and Tricks

To disable the PCI and Suspend IPA Offload and put the NAD into lowest Current Mode:

echo 1 > /sys/bus/pci/devices/0000\:00\:00.0/remove

echo 1 > /sys/kernel/debug/eth/suspend_ipa_offload

To identify the sleep status through software:

cat /sys/power/system_sleep/stats

During each suspend/resume, the sleep counters should increase to indicate that processor went to suspend.

Sample output:

cat /sys/power/system_sleep/stats

RPM Mode:aosd

count:1

time in last mode(msec):0

time since last mode(sec):606

actual last sleep(msec): 154893

RPM Mode:cxsd

count:1

time in last mode(msec):0

time since last mode(sec):606

actual last sleep(msec):154893

RPM Mode:ddr

count:1

time in last mode(msec):0

time since last mode(sec):606

actual last sleep(msec): 154893

To identify the sleep status from each core

cat /sys/power/rpmh_stats/master_stats

During suspend/resume, the sleep counter from each core should increase to indicate that core has agreed to suspend

© Continental Automotive GmbH 2019
Confidential and Proprietary

Sample output:

cat /sys/power/rpmh_stats/master_stats

APSS

Version:0x1

Sleep Count:0x1

Sleep Last Entered At:0x28f8c3579

Sleep Last Exited At:0x29cd7729f

Sleep Accumulated Duration:0xd4b3d26

MPSS

Version:0x1

Sleep Count:0x0

Sleep Last Entered At:0x0

Sleep Last Exited At:0x0

Sleep Accumulated Duration:0x0

8 References

8.1 Abbreviations/Acronyms/Definitions

Not Applicable

8.2 Platform Documentation

TP_102G-00001: Telematics Platform GlossaryTP_F101U-00030: Security User Guide

TP_F101U-00036: NADIF for 5G NAD User Guide

TP_F103U-00034: 5G NAD Module User Guide

5G NAD Module spec: TP_LTE_5G_SA515M_NAD_Specification_v0.5.pdf

8.3 Website links

8.3.1 systemd

General systemd information: https://www.freedesktop.org/wiki/Software/systemd/

systemd man pages: https://www.freedesktop.org/software/systemd/man/systemd.service.html

systemd unit configuration: https://www.freedesktop.org/software/systemd/man/systemd.unit.html

8.3.2 sysfs

General sysfs information: https://www.kernel.org/doc/Documentation/filesystems/sysfs.txt

sysfs gpio kernel documentation: https://www.kernel.org/doc/Documentation/gpio/sysfs.txt

Detailed sysfs class, device, driver documentation: https://www.kernel.org/doc/Documentation/ABI/stable/

© Continental Automotive GmbH 2019
Confidential and Proprietary

Controlling GPIO Hardware from C or C++ (mainly focused on Raspberry Pi, but has general example): https://www.ics.com/blog/how-control-gpio-hardware-c-or-c

© Continental Automotive GmbH 2019
Confidential and Proprietary

9 Revision History

Version	Date	Description
01.00	Nov 18, 2019	Initial ConMod release
02.00	Jul 26, 2022	Added HAL API supported resume reasons
03.00	Apr 15, 2025	Changed section 3.5.1.2

© Continental Automotive GmbH 2019
Confidential and Proprietary